Week 8
Neural Networks For Sequences

Nebius Academy

Plan

- Word Embeddings
  - One-Hot Encoding, Bag of Words
  - Word2Vec and its variations
- Recurrent Neural Networks (RNNs)
  - Recurrent Layer
  - Training of RNNs
  - Language Modelling with RNNs
  - LSTM, GRU
- Practice!

Word Embeddings

Word Embeddings

How do we represent texts in computers?

Texts consist of words. How do we represent words?

One-hot Encoding

Create a vocabulary of a fixed size, e.g. $n = 50.000$ words

1. cat
2. dog
3. …

…

i. mother

…

One-hot Encoding

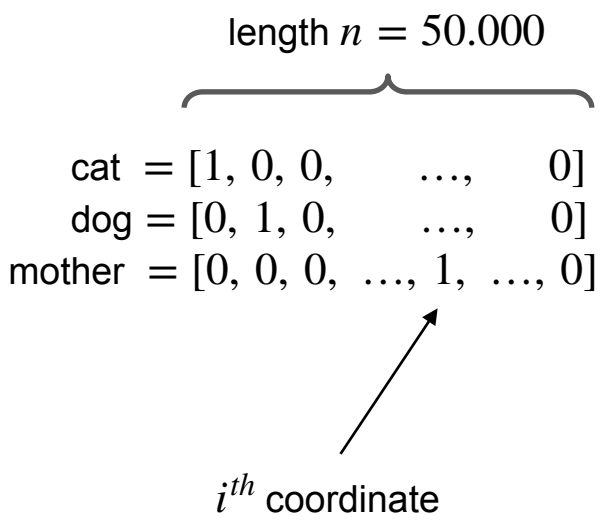
Create a vocabulary of a fixed size, e.g. $n = 50.000$ words

1. cat
2. dog
3. …

…

i. mother

…



One-hot Encoding

Disadvantages:

- One-hot vectors do not reflect meaning of the word

  (we can’t compare two word vectors by meaning)
- Vectors are sparse, they require much additional memory
- Vocabulary size is fixed. Words outside of vocabulary cannot be processed
- If vocabulary changes, one-hot vectors should be re-calculated

Bag of Words (BoG)

Bag of Words is a way to create text vectors based on one-hot encodings of its words

Text vector is a simple sum of its word vectors

Bag of Words (BoG)

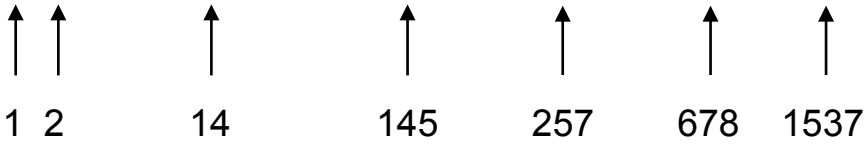
1. a
2. and
…
14. are
…
145. cat
…
257. dog
…
678. is
…
1537. sleeping
…

1.a cat and a dog are sleeping

2.a dog is walking

BoW for these sentences:

1. [2, 1, 0, … 1, 0, … 0, 1, 0, … 1, 0, … 0, … 1, … 0]
2. [1, 0, 0, … 0, 0, … 0, 0, 0, … 1, 0, … 1, … 0, … 0]



Bag of Words (BoG)

Bag of Words inherits all the disadvantages of One-hot Encoding:

- Text vectors reflect meaning of the text really poorly. Word order is not considered, words can have different importance.
- Vectors are sparse, they require much additional memory
- Vocabulary size is fixed. Words outside of vocabulary cannot be processed
- If vocabulary changes, BoW vectors should be re-calculated

Other types of word vectors

One-Hot Encoding and Bag of Words are quite simple and weak representations of words/texts. There are more sophisticated ideas on how to construct more representative word vectors. For example:

- TF-IDF
- Latent Semantic Analysis (LSA)

Contextual Embeddings

Look at these three sentences:

1. Maria drives a _____
2. A wheel of a _____ was pierced
3. _____ has both its wheels framed

Which words can be placed on placeholders?

	1	2	3
Bicycle	+	+	+
Motorbike	+	+	+
Car	+	+	-
Horse	+	-	-

Contextual Embeddings

Idea:

A meaning of the word is determined by a context where it can be used.

Let's build word vectors based on their context.

A bicycle has a nice frame coloured in blue

Sliding window of size=5

A matrix of word co-occurences in the corpus of texts:

	a	horse	ride	bicycle	frame	rose
a		256	45	230	90	134
horse	256		137	4	2	5
ride	45	137		120	34	3
bicycle	230	4	120		76	2
frame	90	2	34	76		0
rose	134	5	3	2	0	

Contextual Embeddings

Idea:

A meaning of the word is determined by a context where it can be used.

	a	horse	ride	bicycle	frame	rose
a		256	45	230	90	134
horse	256		137	4	2	5
ride	45	137		120	34	3
bicycle	230	4	120		76	2
frame	90	2	34	76		0
rose	134	5	3	2	0	

Word vector is a row of the matrix

Contextual Word Vectors

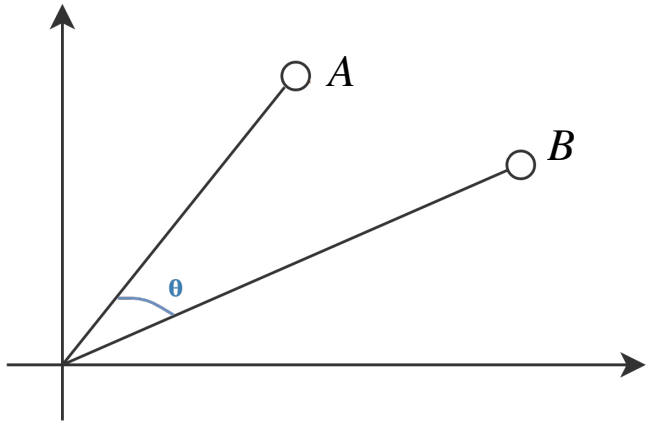
Advantages:

- Vectors do reflect word meanings. They can be compared by dot product/ cosine similarity

Limitations:

- Vectors are still sparse and require much additional memory
- Vocabulary size is limited. Words out of vocabulary cannot be processed
- If vocabulary changes, vectors should be re-calculated
- Vectors of rare words are not quite informative

Cosine similarity

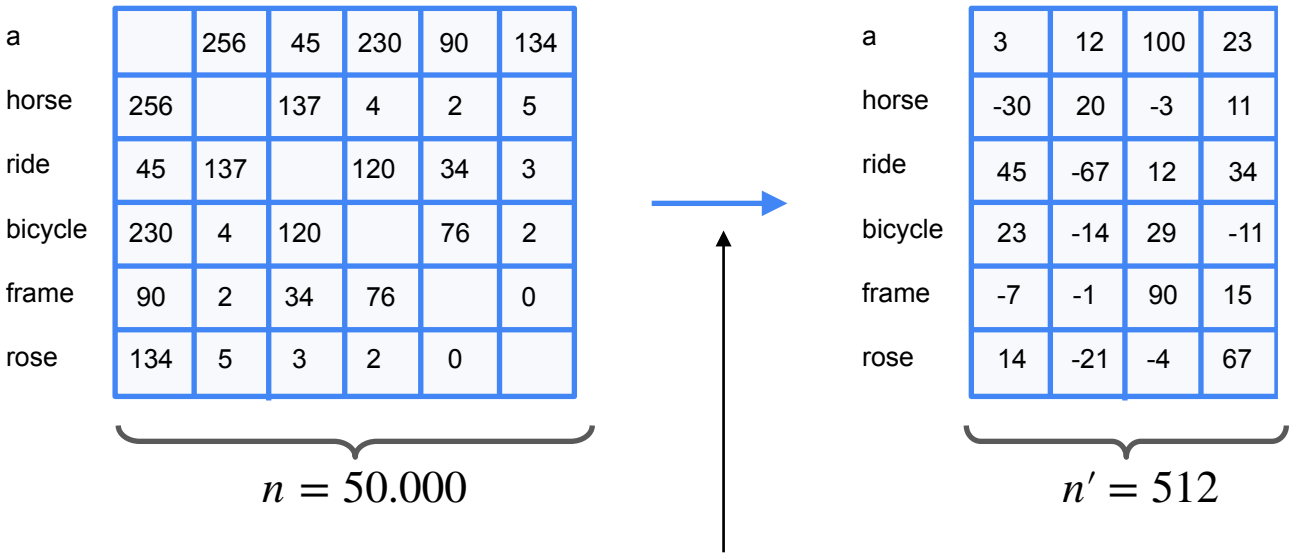


Dot product between two vectors: $A \cdot B = \sum_{i=1}^{n} A_i B_i$

Cosine similarity: $sim(A, B) = cos(\theta) = \frac{A \cdot B}{||A|| \, ||B||} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$

Cosine distance $1 - sim(A, B)$

Contextual Embeddings



Word Embeddings

What we did before was that we constructed word/text vectors/matrices based on some ideas of how it can be done so that these vectors/matrices reflect the meanings of words/texts

But what if we try to ***learn*** these vectors instead?

Word Embeddings

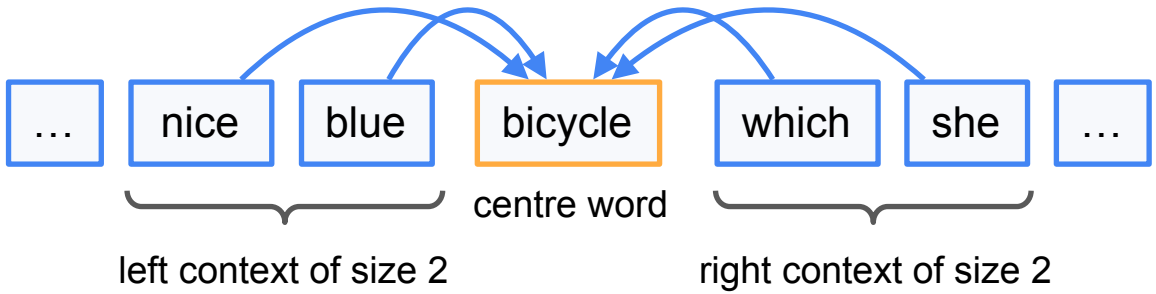
So we want to learn vectors of small dimensionality which would reflect meanings of words, i.e. we would be able to compare these vectors using some distance function (e.g. MSE, cosine distance)

Such learned vectors will be called ***word embeddings***

Word2Vec

We will train a neural network to predict a word based on its context

Maria has a nice blue bicycle which she likes to ride

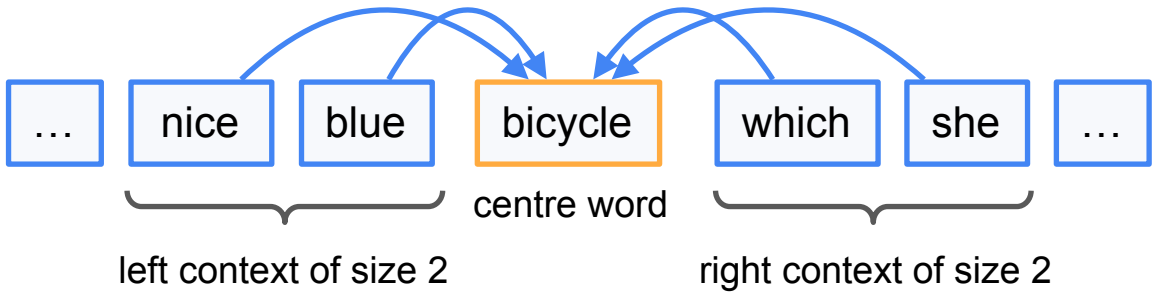


Word2Vec

Our dataset is a collection of texts. We will go through texts with a sliding window of size n and at each step train a network to predict a centre word based on its context.

We can choose $n = 5$

Maria has a nice blue bicycle which she likes to ride

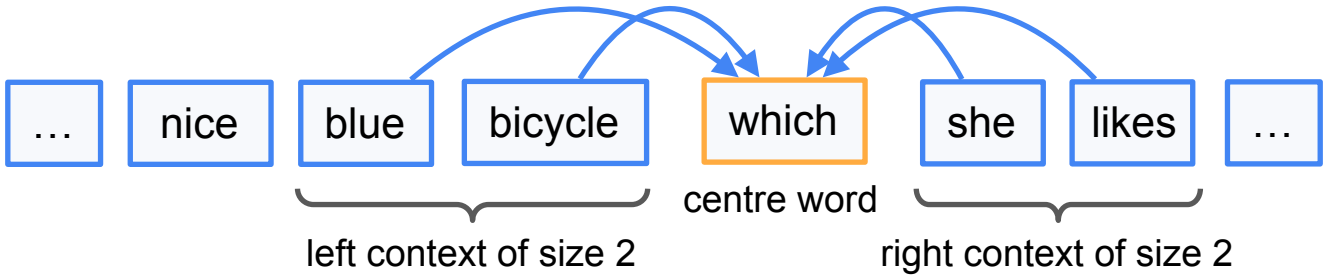


Word2Vec

Our dataset is a collection of texts. We will go through texts with a sliding window of size n and at each step train a network to predict a centre word based on its context.

We can choose $n = 5$

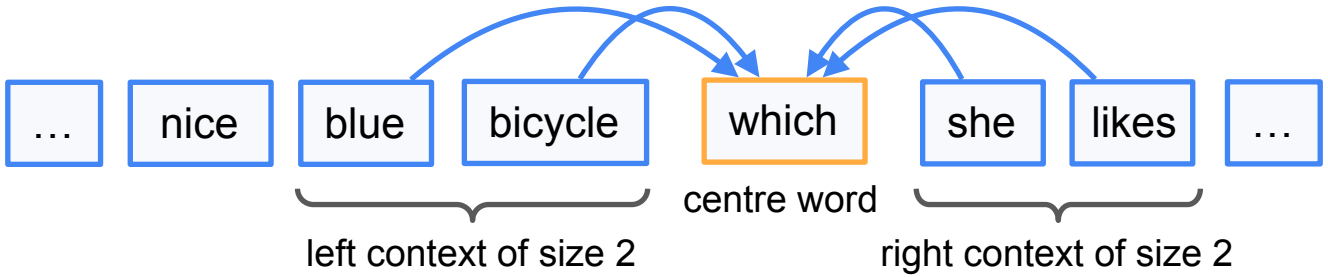
Maria has a nice blue bicycle which she likes to ride



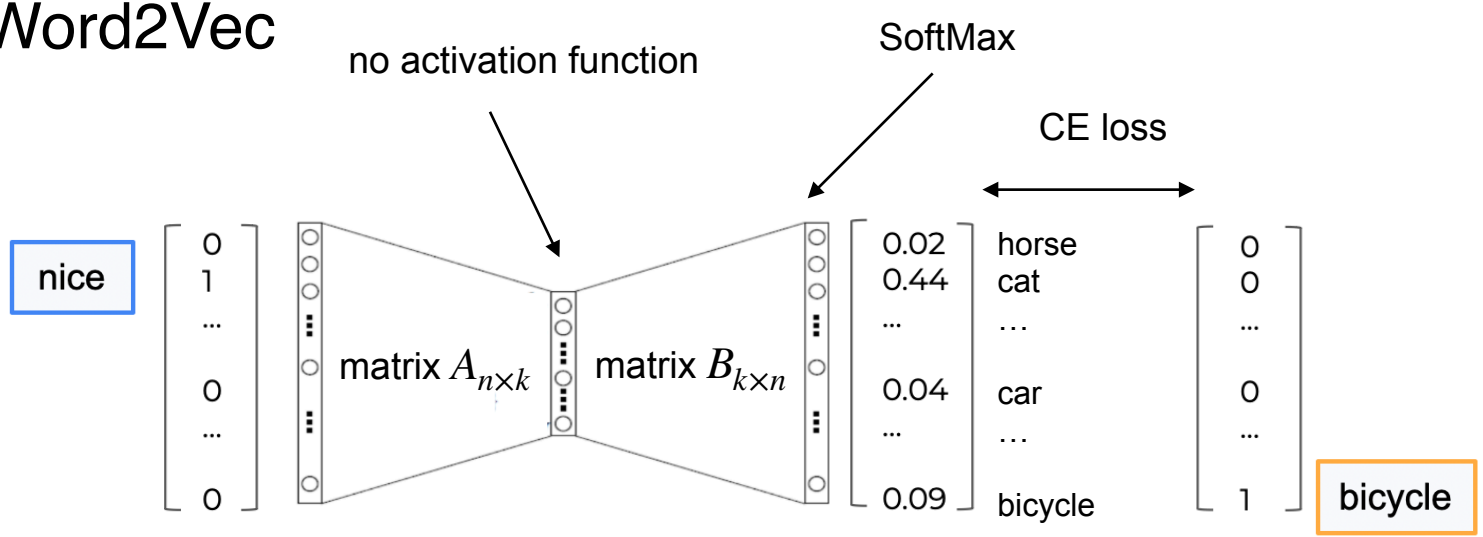
Word2Vec

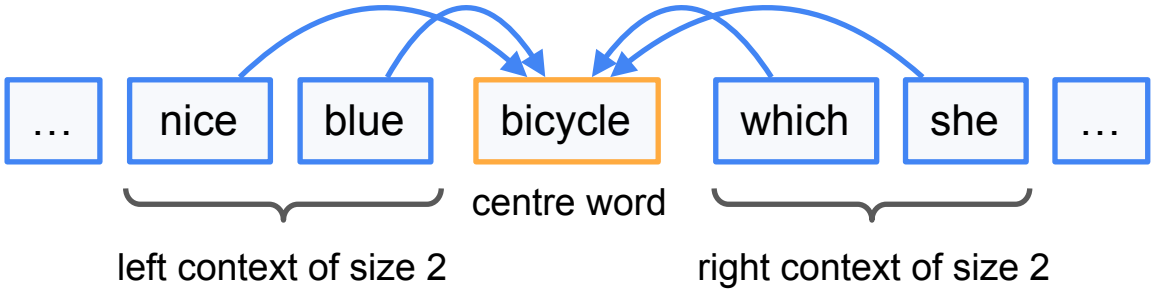
Let’s formalise our task:

- We have a classification task with number of classes equal to the vocabulary size
- Model takes a context word as input, and should predict probability distribution over vocabulary words.

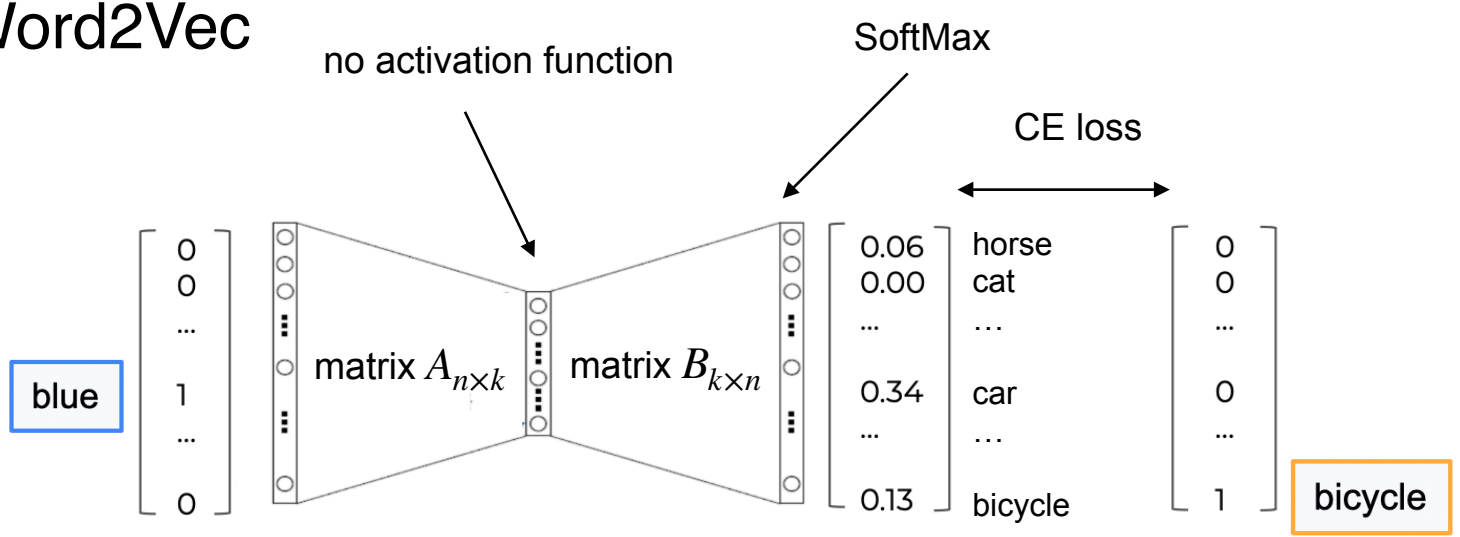


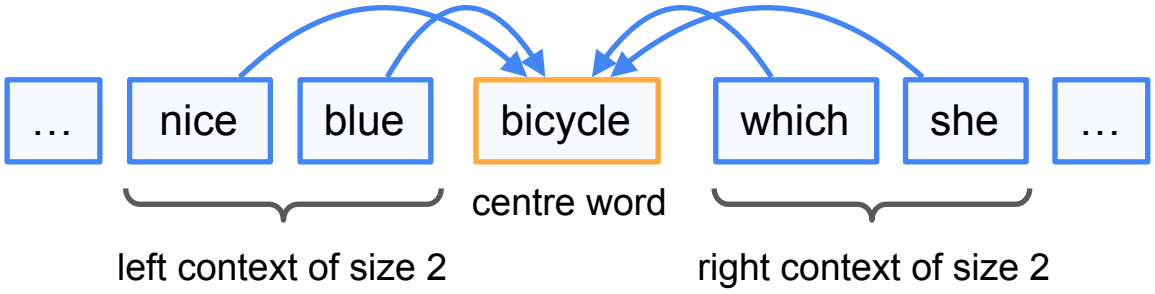
Word2Vec


no activation function
SoftMax
CE loss
nice
0
1
...
0
...
0
matrix $A_{n\times k}$
matrix $B_{k\times n}$
0.02
0.44
...
0.04
...
0.09
horse
cat
...
car
...
bicycle
0
0
...
0
...
1
bicycle

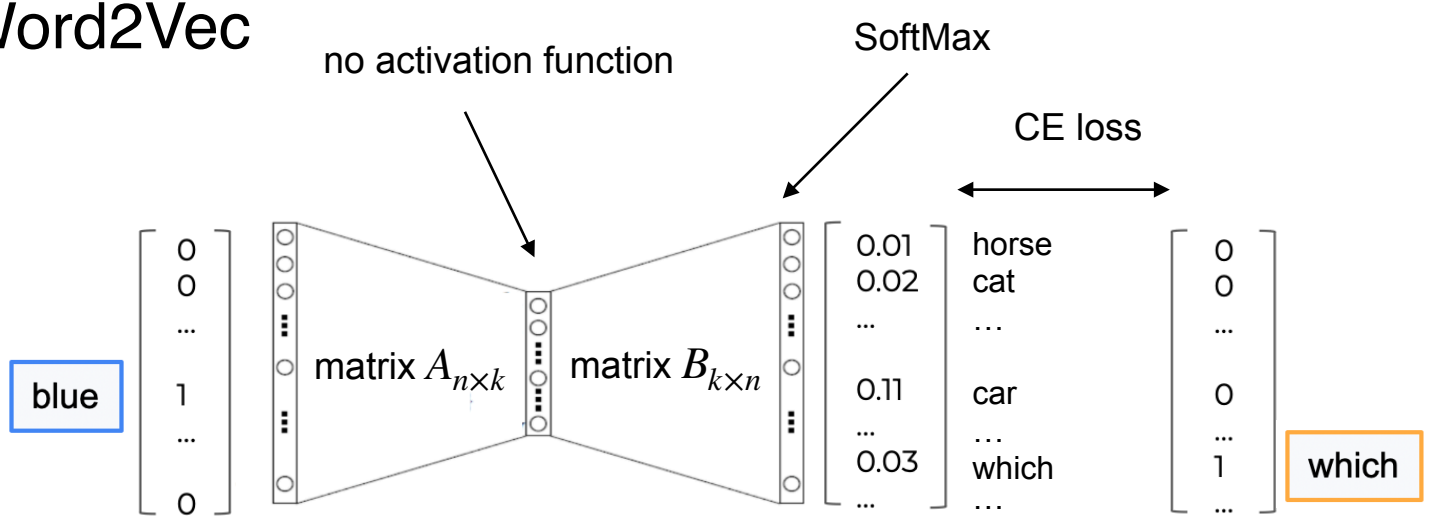

...
nice
blue
bicycle
which
she
...
centre word
left context of size 2
right context of size 2

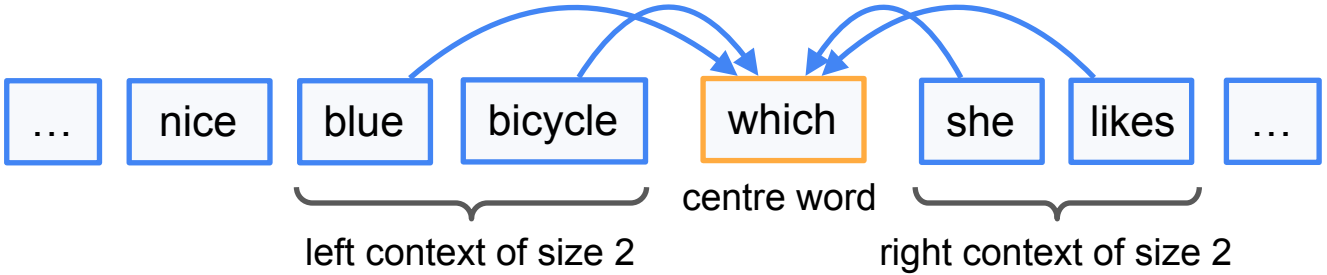
Word2Vec


no activation function
SoftMax
CE loss
blue
0
0
...
1
...
0
matrix $A_{n\times k}$
matrix $B_{k\times n}$
0.06 horse
0.00 cat
...
0.34 car
...
0.13 bicycle
0
0
...
0
...
1
bicycle

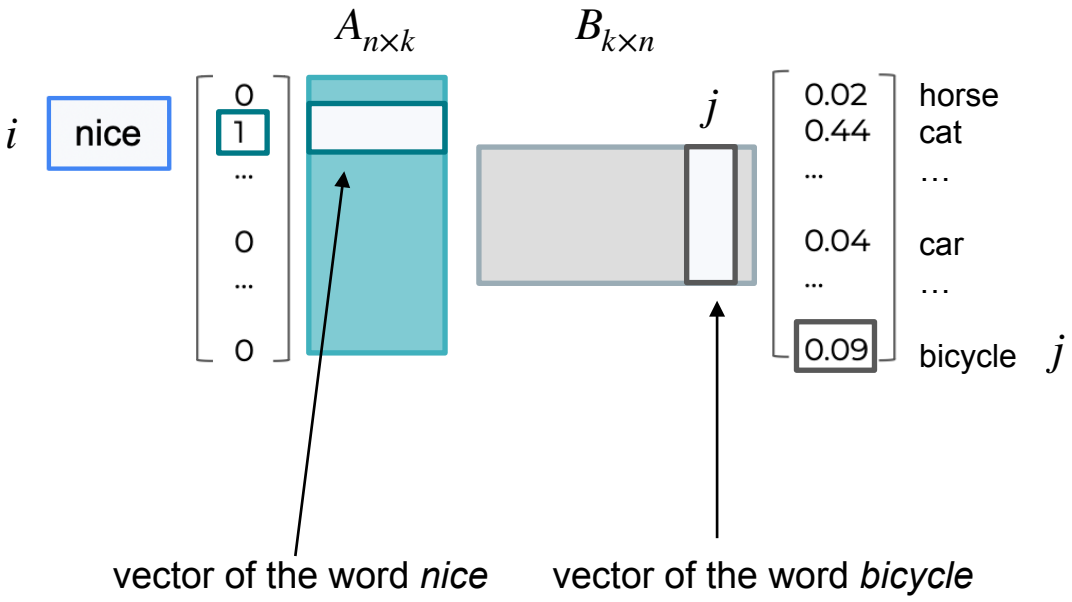

…
nice
blue
bicycle
which
she
…
centre word
left context of size 2
right context of size 2

Word2Vec


no activation function
SoftMax
CE loss
blue
0
0
...
1
...
0
matrix $A_{n\times k}$
matrix $B_{k\times n}$
0.01
0.02
...
0.11
...
0.03
...
horse
cat
...
car
...
which
...
0
0
...
0
...
1
...
which


…
nice
blue
bicycle
which
she
likes
…
centre word
left context of size 2
right context of size 2

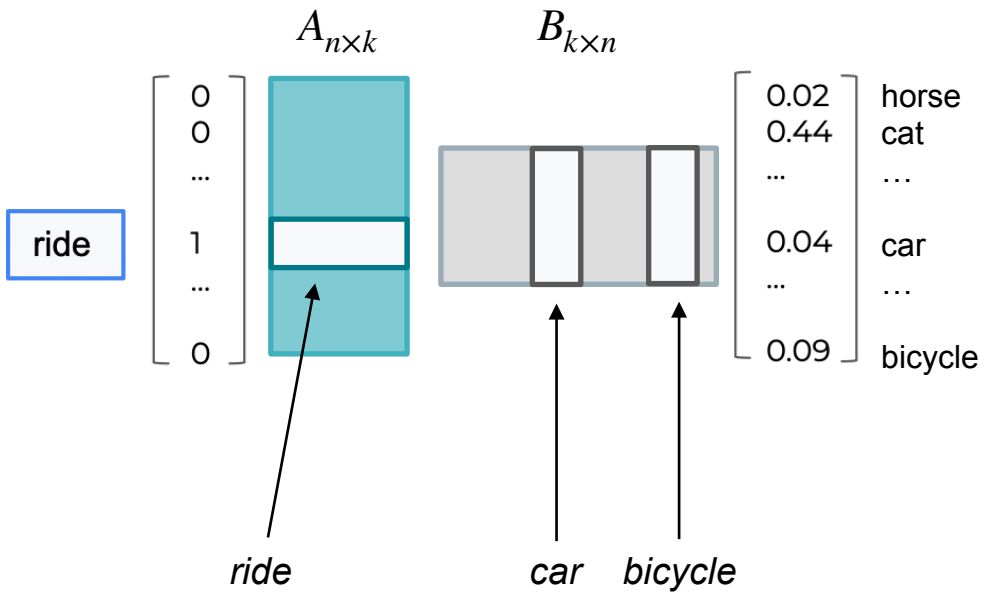
Word2Vec



Logit corresponding to the word *bicycle* is a dot product between i^{th} row of matrix A and j^{th} column of matrix B

So during training, we maximise the dot product between vectors of the words frequently used together.

Word2Vec



$$dot\,(ride, car) \rightarrow max$$
$$dot\,(ride, bicycle) \rightarrow max$$

$$\downarrow$$

$$dot\,(car, bicycle) \rightarrow max$$

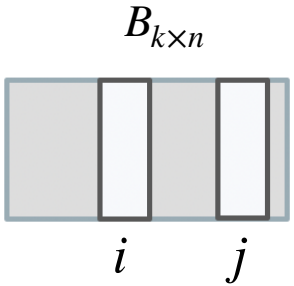
Word2Vec

After training, we get the embedding of a size k for every word in the vocabulary.

k can be chosen arbitrary.

These embeddings reflect the meaning of words.

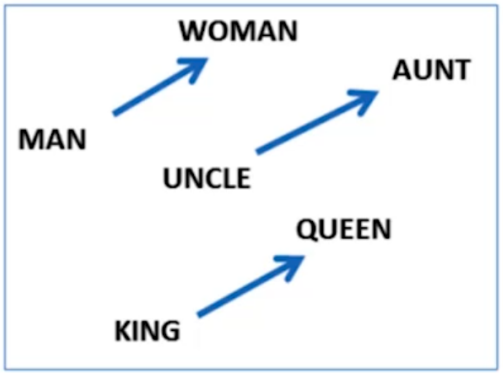
We can compare these embeddings using dot product (cosine similarity)

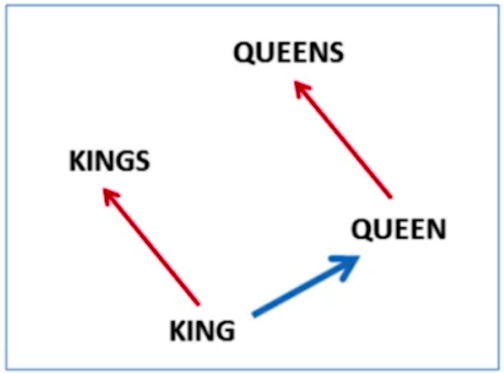


Word2Vec

Vector arithmetic can be executed on Word2Vec embeddings

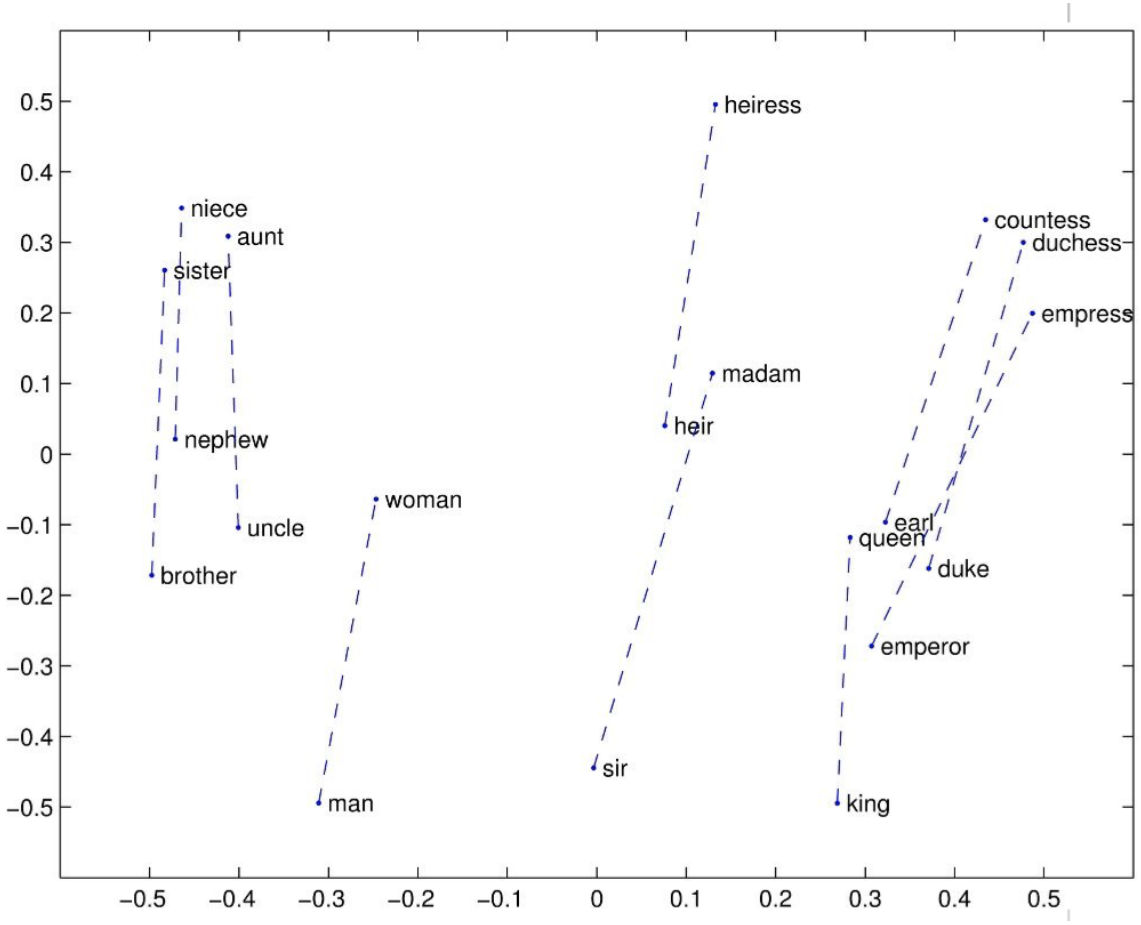
$$v(king) + (v(woman) - v(man)) \sim v(queen)$$



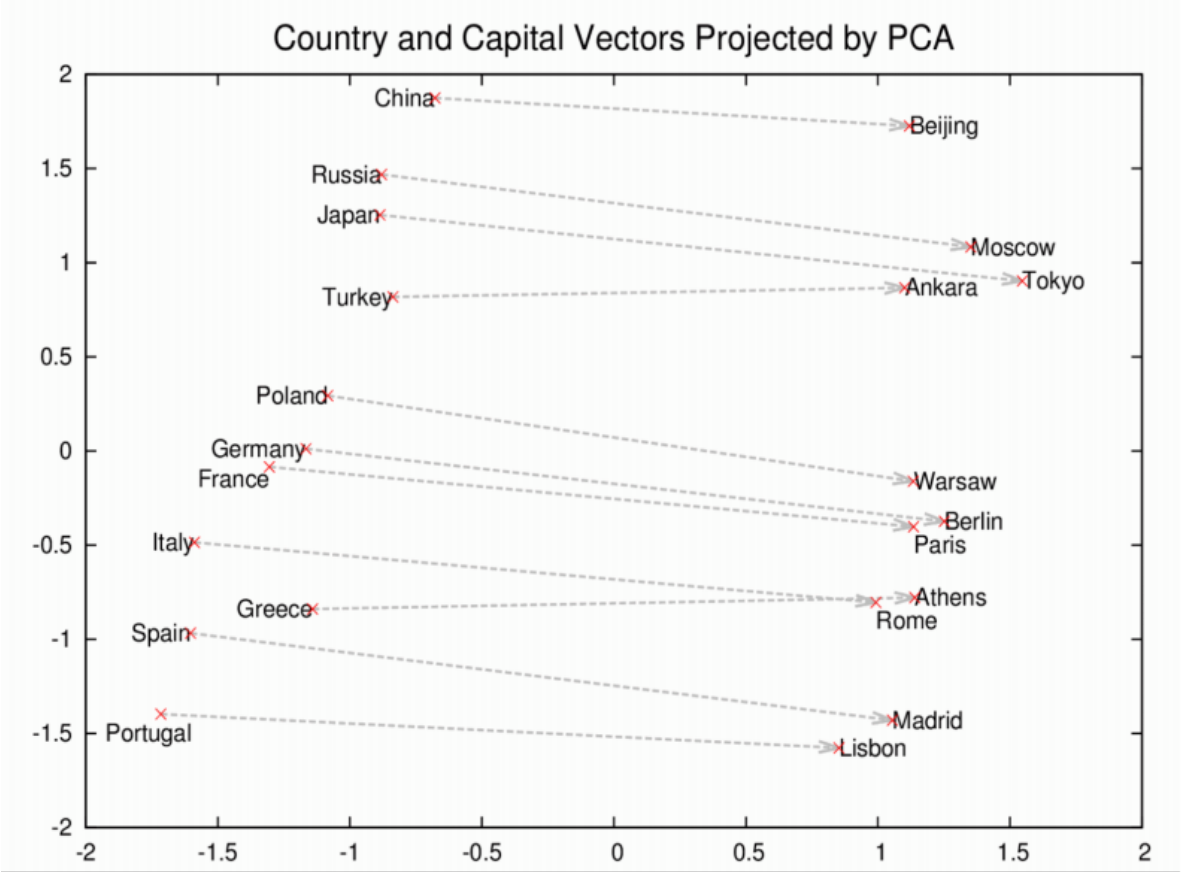


Word2Vec embeddings projected onto 2-d place by PCA

Word2Vec


0.5
0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.4
−0.5
−0.5
−0.4
−0.3
−0.2
−0.1
0
0.1
0.2
0.3
0.4
0.5
niece
aunt
sister
nephew
uncle
brother
woman
man
heiress
madam
heir
sir
countess
duchess
empress
earl
queen
duke
emperor
king

Word2Vec


Country and Capital Vectors Projected by PCA
China
Beijing
Russia
Moscow
Japan
Tokyo
Turkey
Ankara
Poland
Warsaw
Germany
Berlin
France
Paris
Italy
Athens
Greece
Rome
Spain
Madrid
Portugal
Lisbon
2
1.5
1
0.5
0
-0.5
-1
-1.5
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2

Word2Vec

Advantages:

- Vectors do reflect word meanings. They can be compared by dot product
- Dimensionality of vectors does not depend on vocabulary size
- If vocabulary changes, vectors can be fine-tuned

Limitations:

- Vocabulary size is limited. Words out of vocabulary cannot be processed
- Vectors of rare words are not quite informative
- Words having common root are processed independently

eat, eater, eating

FastText

Idea: let's divide words into parts and build vectors for these parts.

There might be different approaches to dividing words into parts:

- Simply by n-grams: *eating* $=<$ ea*, ati, tin, ng* $>$
- Byte-pair encoding (BPE): *eating* $=<$ *eat, ing* $>$

We will train embeddings for all parts, and then calculate word vector as simple sum of vectors of its parts:

$v(\textit{eating}) = v(\textit{eat}) + v(\textit{ing})$

GloVe (Global Vectors)

GloVe uses statistical information about the frequency of words and phrases in text to improve the learning of rare word embeddings.

More can be found here

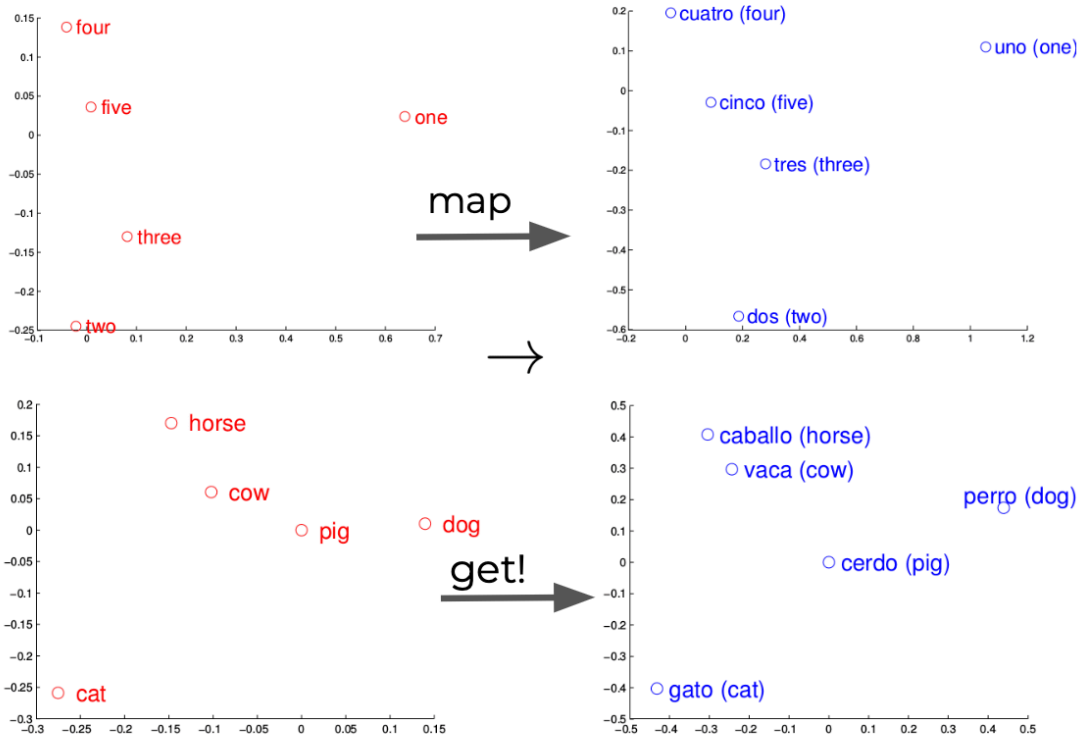
Language Map

Let's say we have texts in an unknown language that we want to understand.

How can this be done:

- Learn word embeddings for English words
- Learn word embeddings for an unknown language
- Find a transformation f that translates English word embeddings into word embeddings of an unknown language.

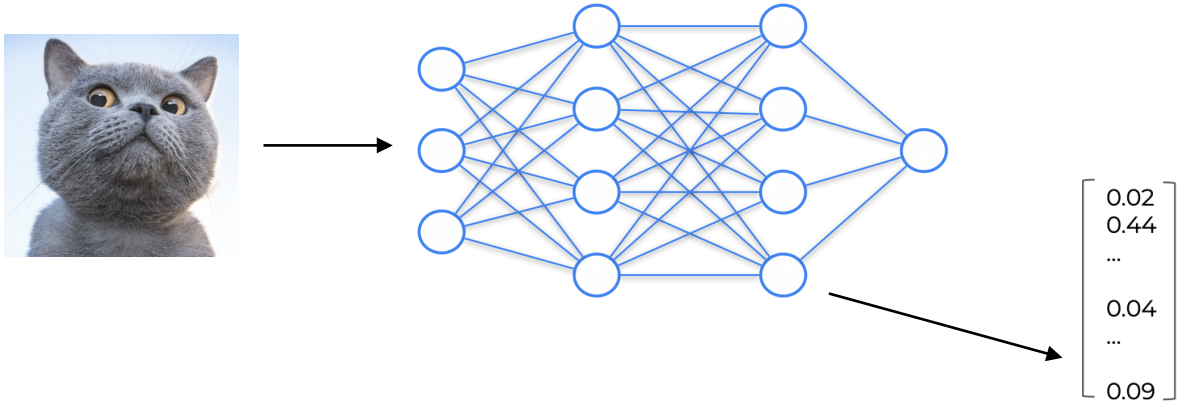
Language Map



General Notion of Embeddings

Embedding is a vector representation of an object which reflects information about an object.

Vector outputs of hidden layers of models can also be considered as embeddings



General Notion of Embeddings

Example: finding similar images

- Get ResNet network pre-trained on ImageNet
- Get embeddings of pre-last layer of the model for each image in dataset
- Given input image X, get its embedding from the model and find images in dataset which have closest embeddings to that of X (closest in terms of MSE/dot product)

Recurrent Neural Networks (RNNs)

Sequence processing

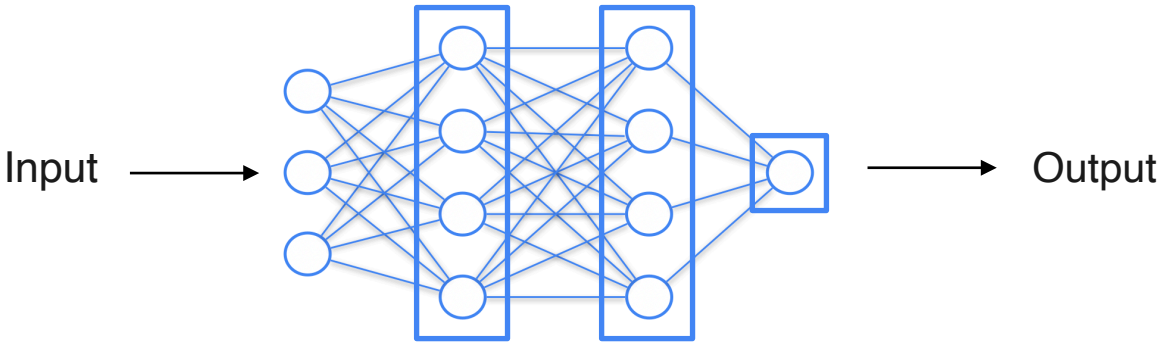
The difference between text/sound and other types of data (for example, images) is the presence of a *time component*.

We read text not instantly, but word by word, in a strictly defined order.

It would be nice to come up with an idea of a neural network that would take into account this feature of these types of data.

Recurrent Layer

How ***fully-connected*** neural network works:

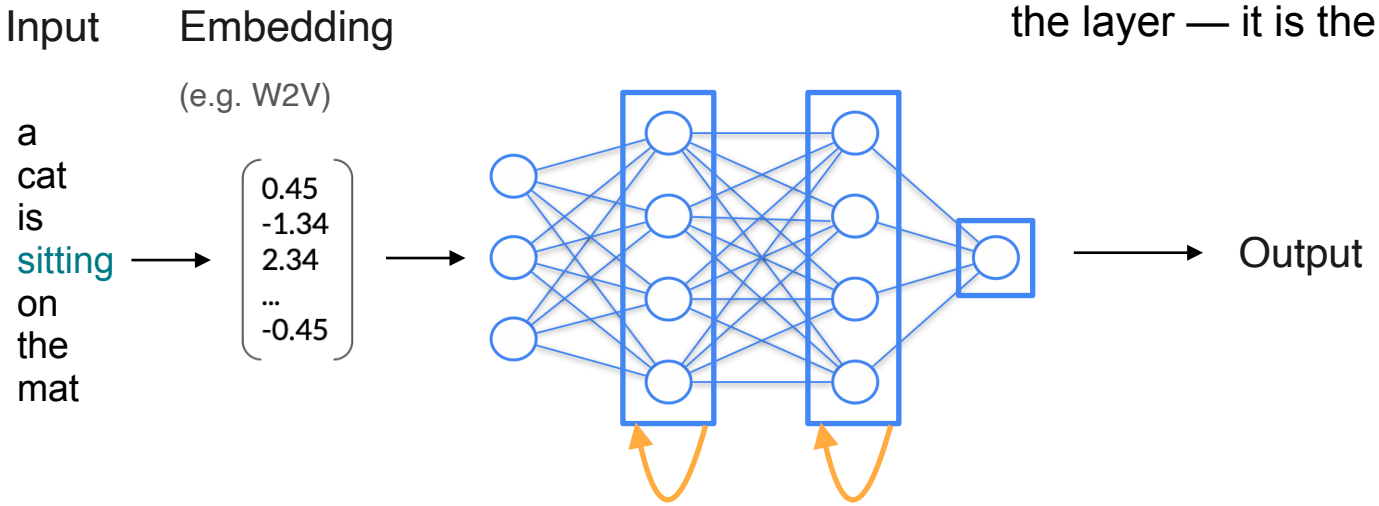


Recurrent Layer

How ***recurrent*** neural network works:

A recurrent neural network processes one token of text at a time.

A “from itself to itself” connection is added to the layer — it is the “memory” of the layer

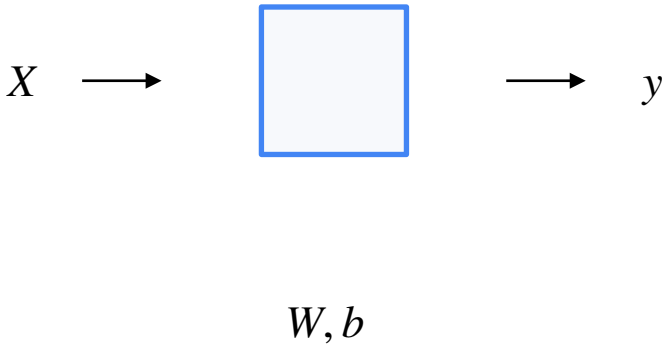


Recurrent Layer

Fully-connected layer:

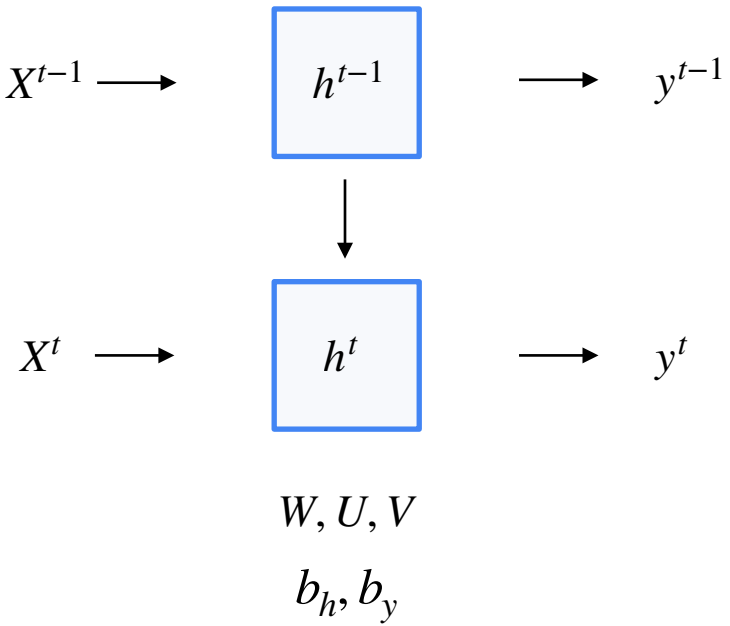
Layer output:

$$y = \sigma(WX + b)$$



Recurrent Layer

Recurrent layer:



Update hidden state:

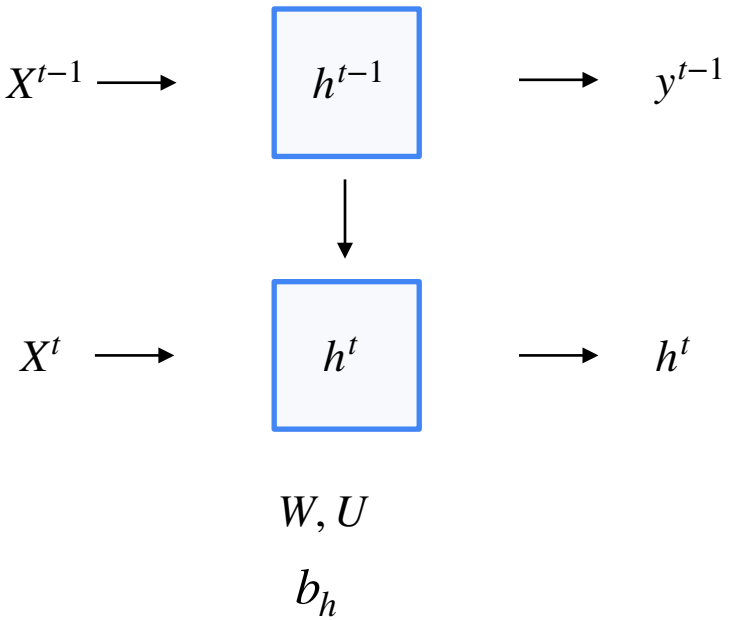
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

Layer output:

$$y^t = \sigma(Vh^t + b_y)$$

Recurrent Layer

Recurrent layer:



Update hidden state:

$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

Layer output:

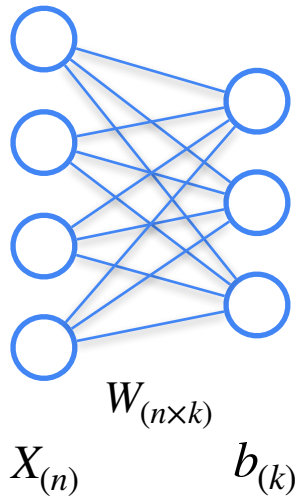
$$y^t = h^t$$

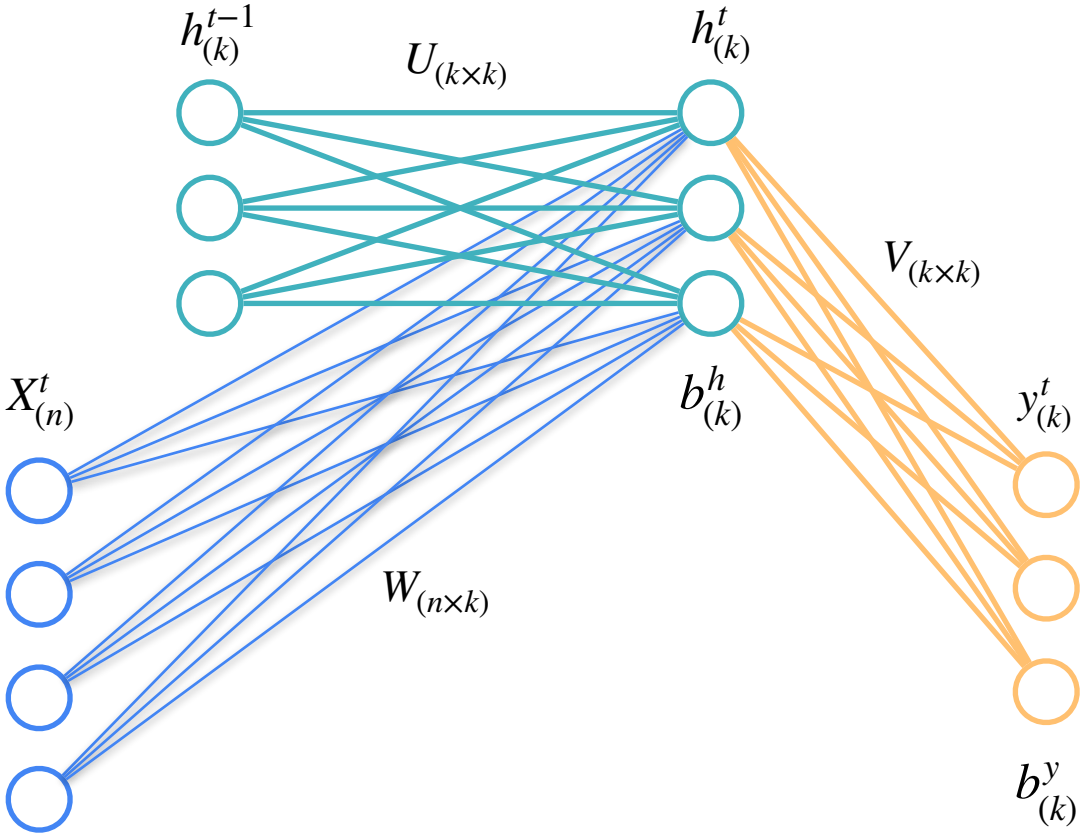
Recurrent Layer

Fully-connected layer:

Layer output:

$$y = \sigma(WX + b)$$





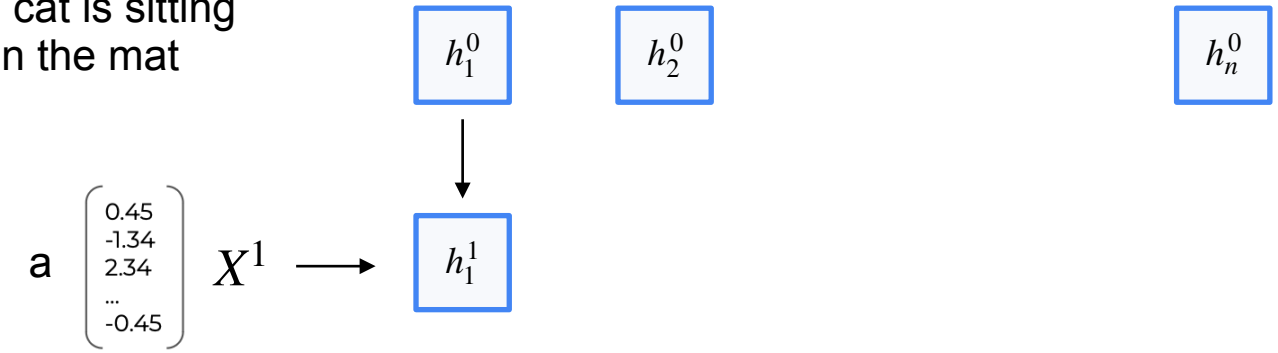
Update hidden state:

$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

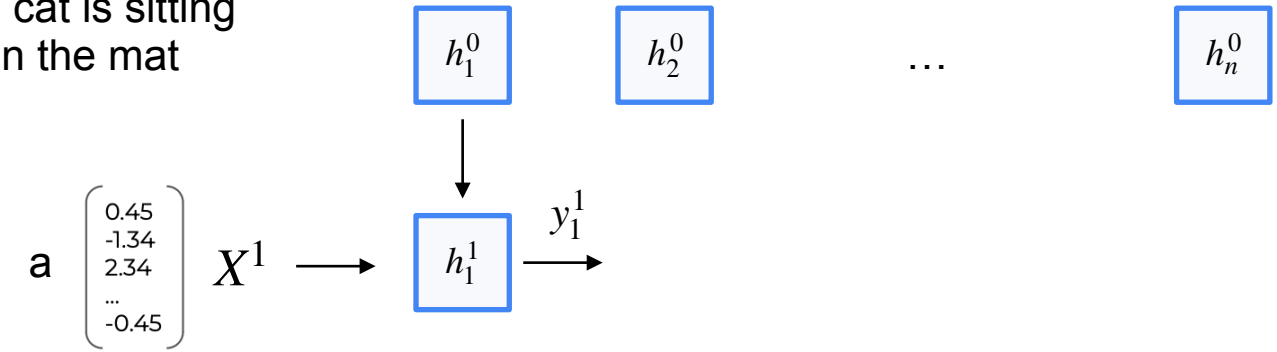
Layer output:

$$y^t = \sigma(Vh^t + b_y)$$

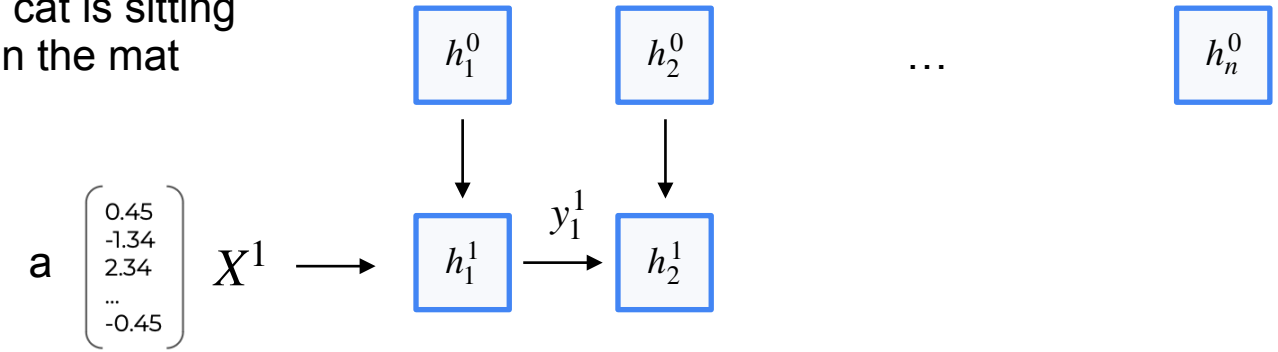
a cat is sitting
on the mat



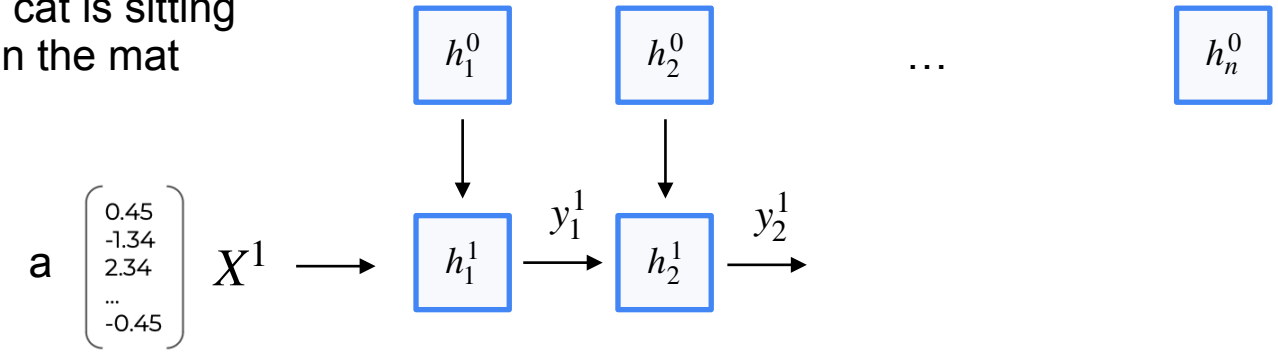
a cat is sitting
on the mat



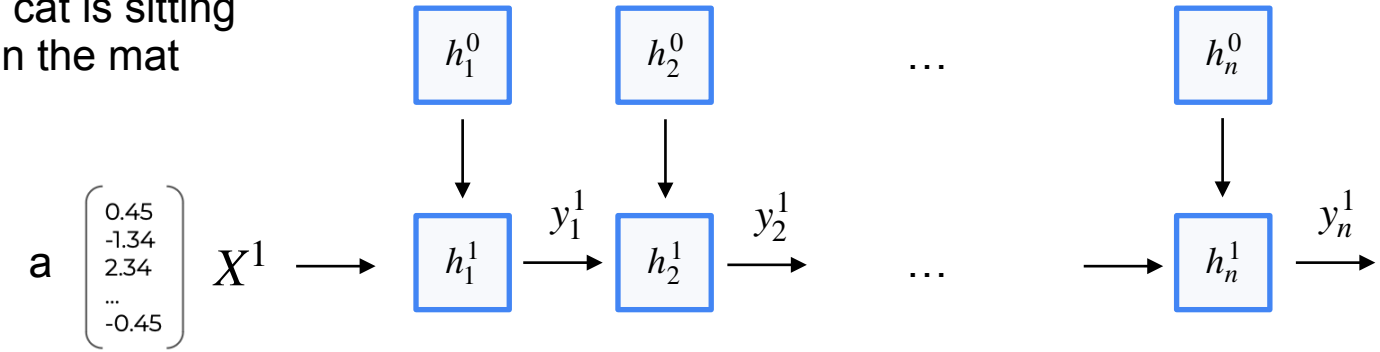
a cat is sitting
on the mat



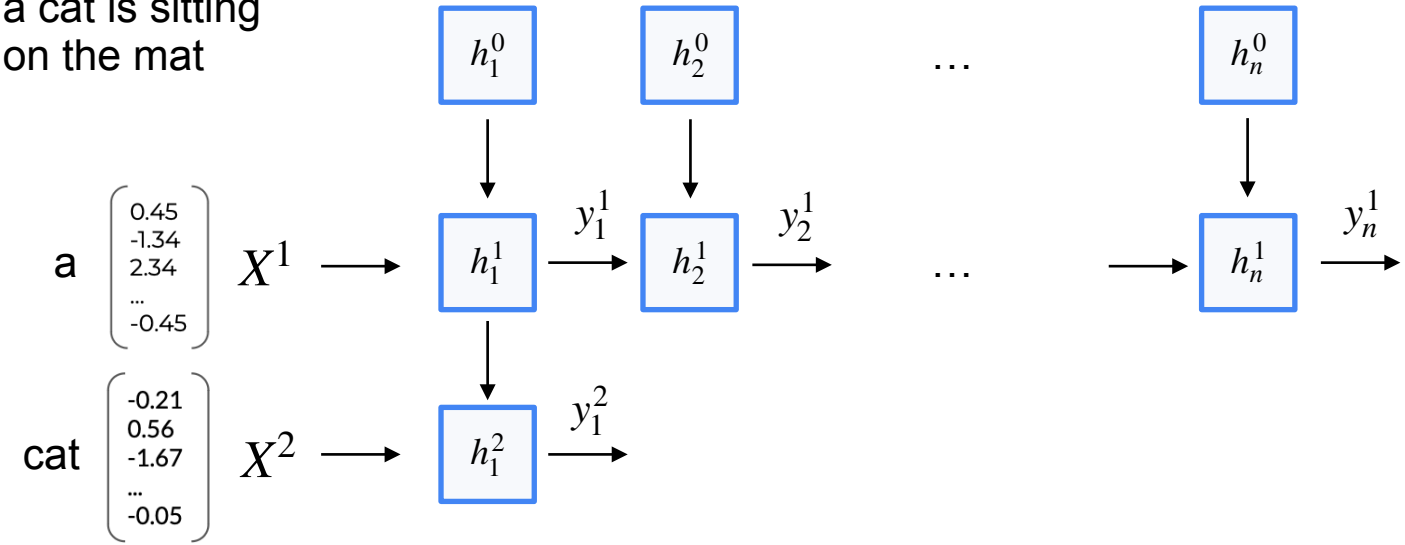
a cat is sitting
on the mat



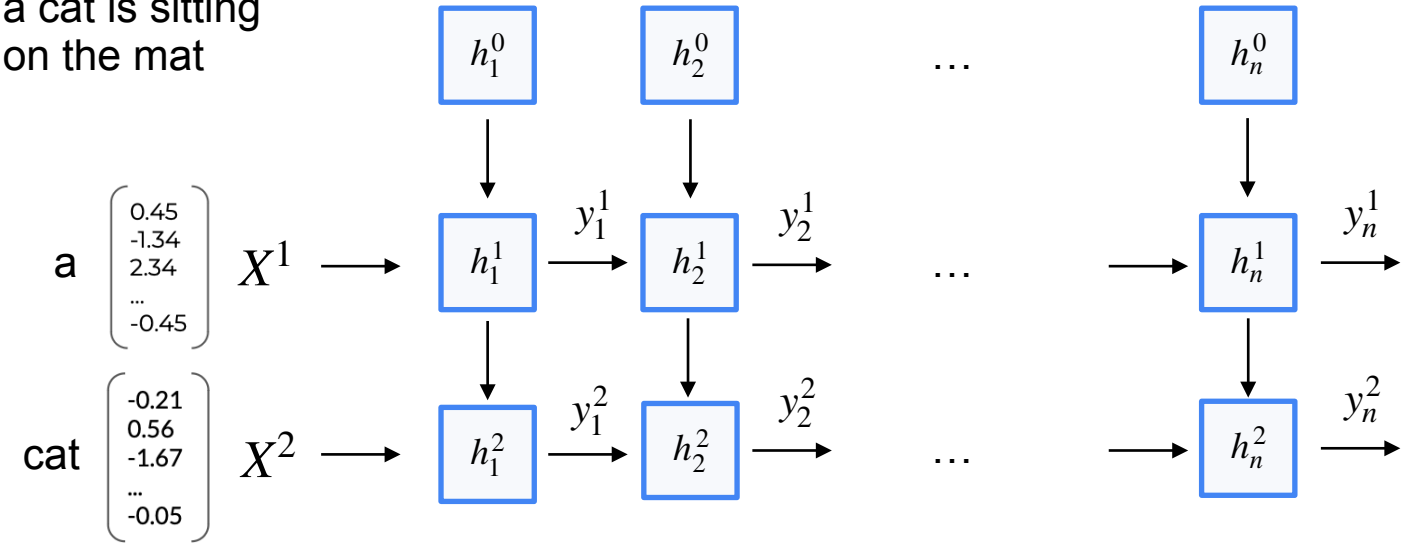
a cat is sitting
on the mat

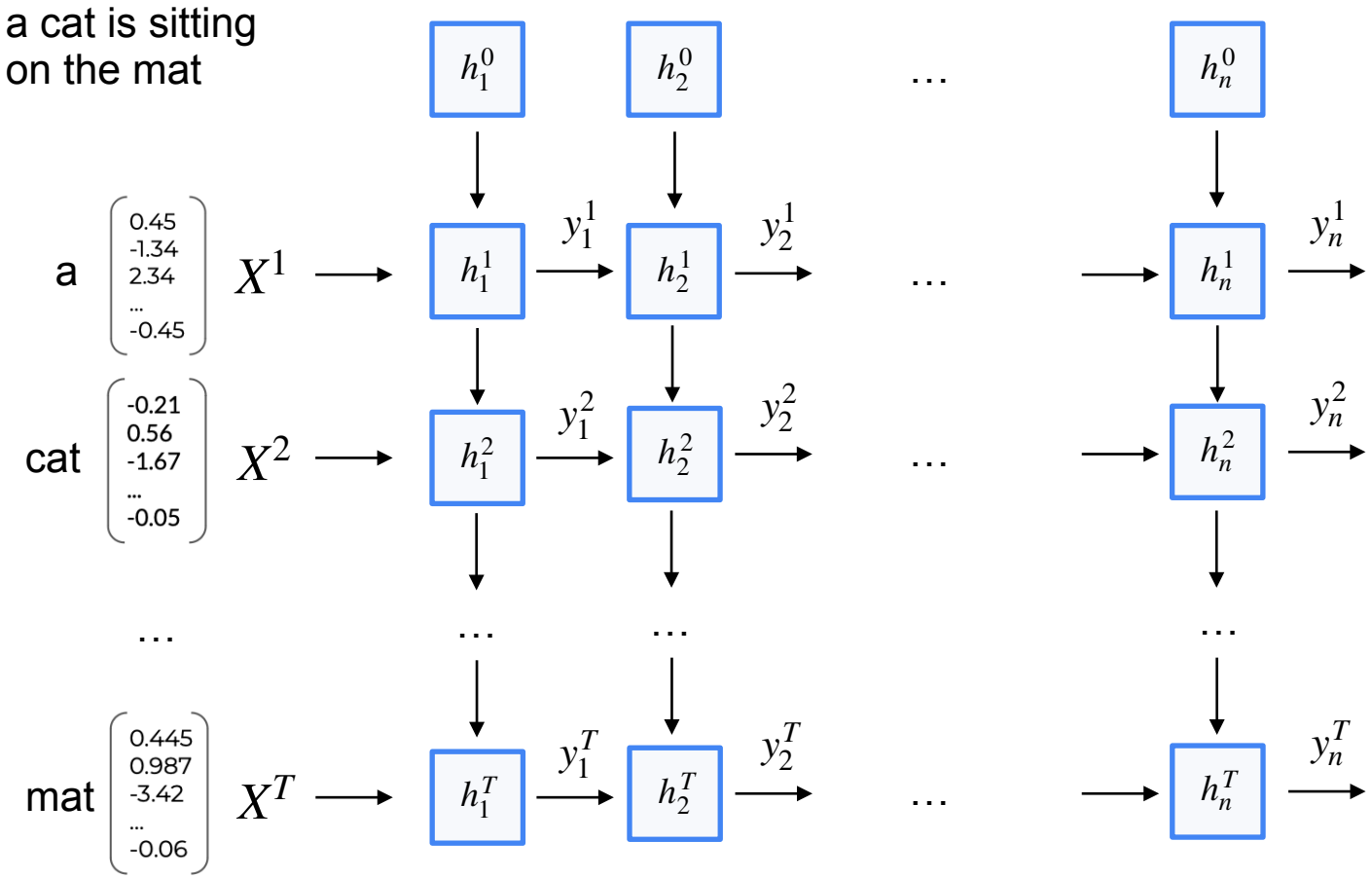


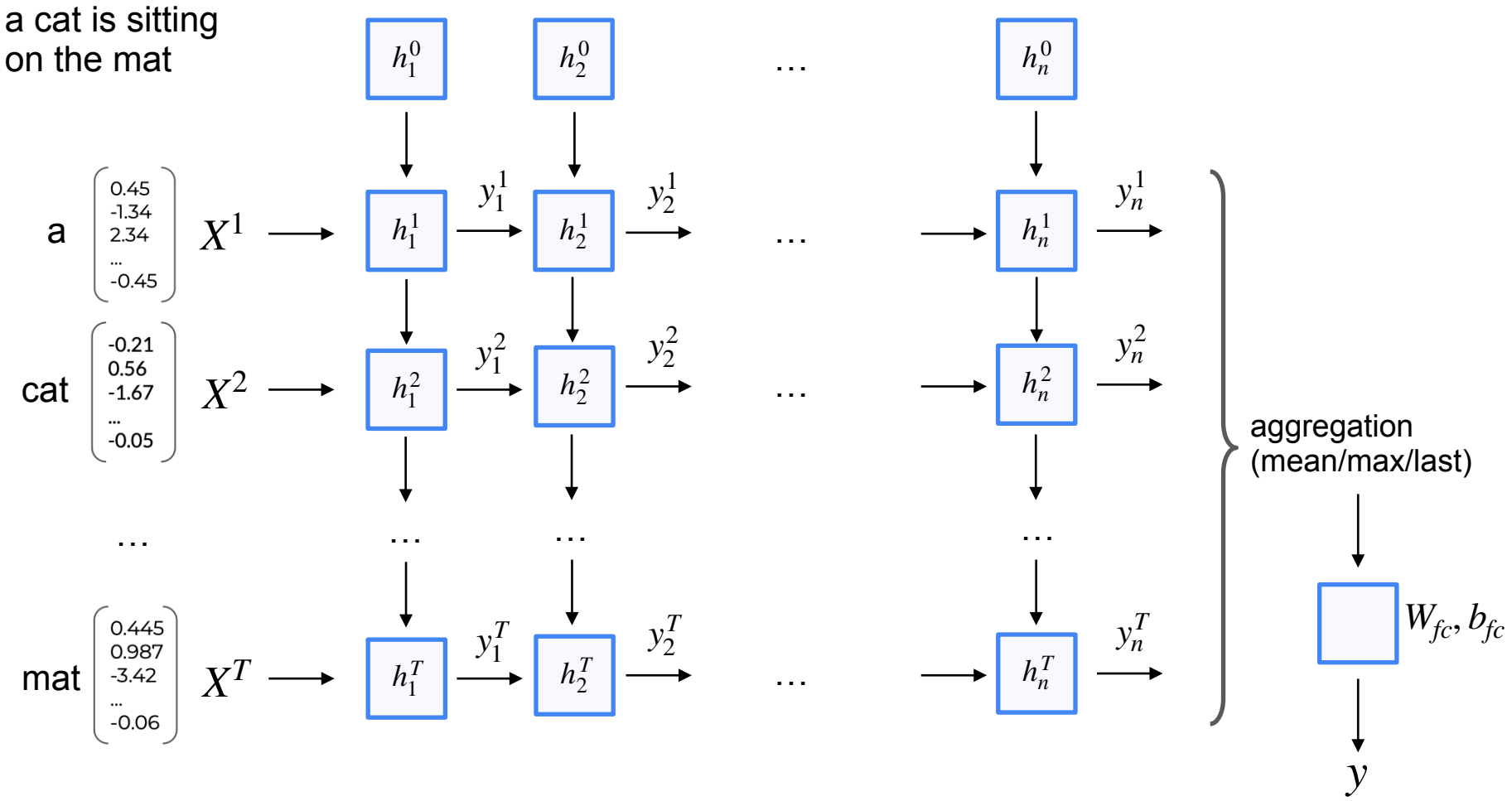
a cat is sitting on the mat

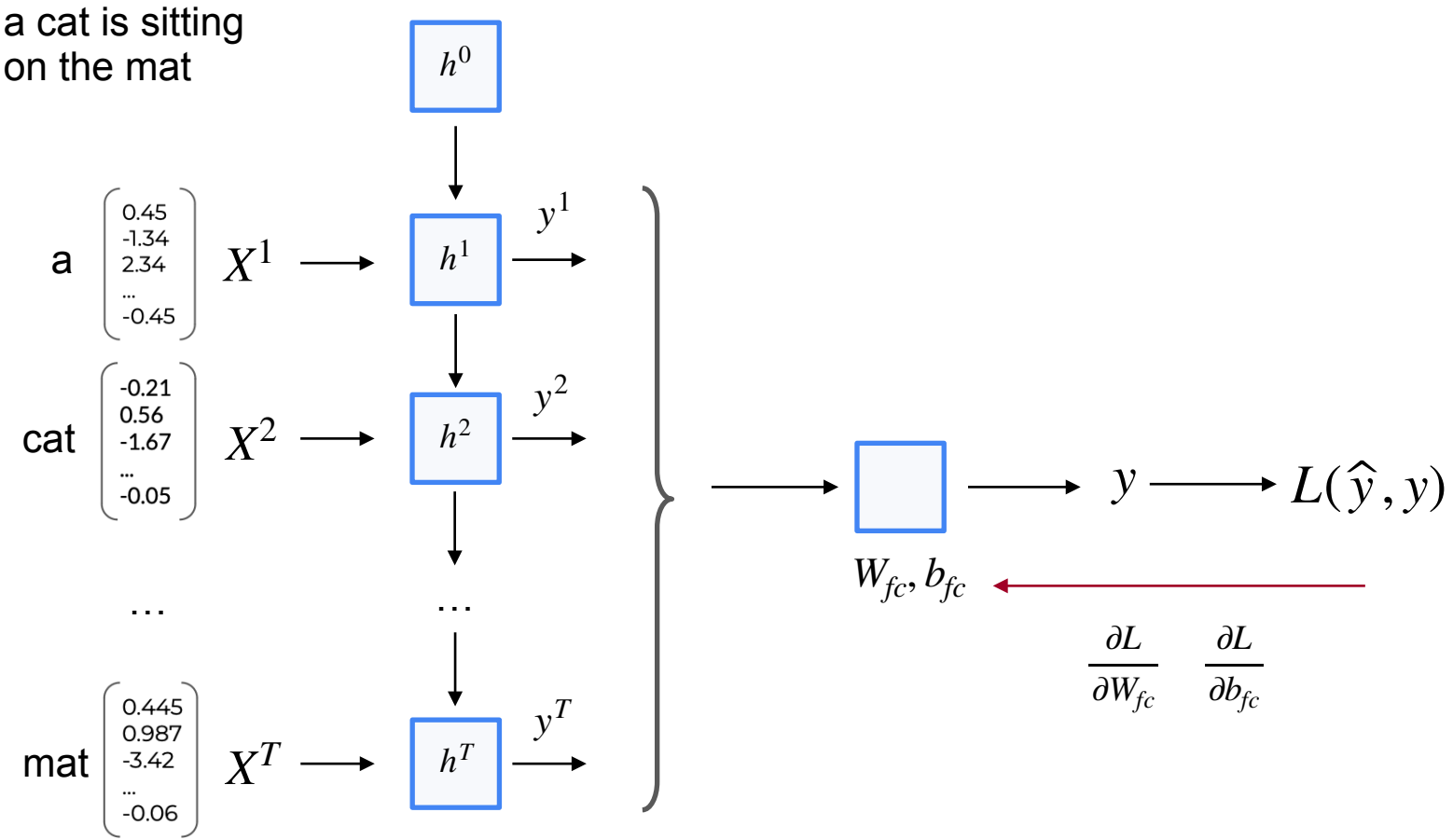


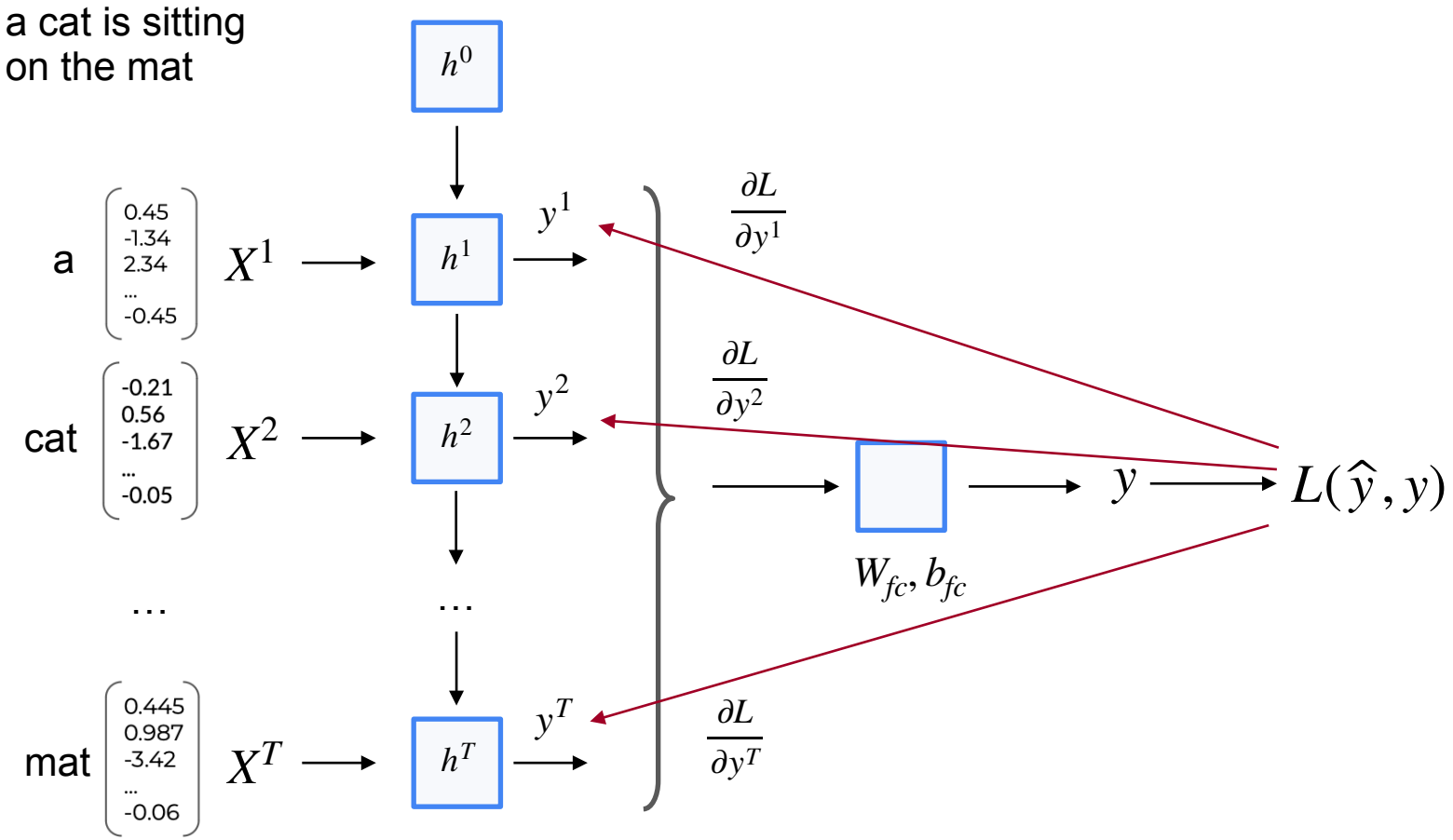
a cat is sitting on the mat

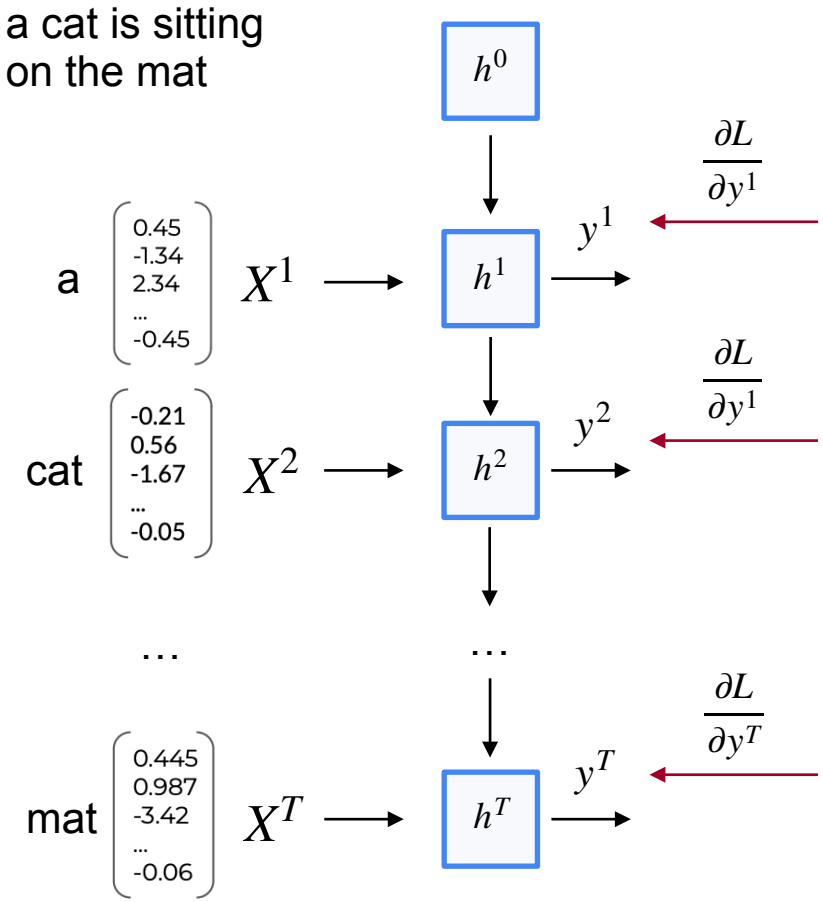



a cat is sitting on the mat
a
cat
mat
0.45
-1.34
2.34
...
-0.45
-0.21
0.56
-1.67
...
-0.05
0.445
0.987
-3.42
...
-0.06
X^1
X^2
X^T
h_1^0 h_2^0 ... h_n^0
h_1^1 y_1^1 h_2^1 y_2^1 ... h_n^1 y_n^1
h_1^2 y_1^2 h_2^2 y_2^2 ... h_n^2 y_n^2
h_1^T y_1^T h_2^T y_2^T ... h_n^T y_n^T


a cat is sitting on the mat
a
cat
mat
0.45
-1.34
2.34
...
-0.45
-0.21
0.56
-1.67
...
-0.05
0.445
0.987
-3.42
...
-0.06
X^1
X^2
X^T
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
h_1^2
h_2^2
h_n^2
h_1^T
h_2^T
h_n^T
y_1^1
y_2^1
y_n^1
y_1^2
y_2^2
y_n^2
y_1^T
y_2^T
y_n^T
aggregation
(mean/max/last)
W_{fc}, b_{fc}
y


a cat is sitting
on the mat
h^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h^1
y^1
cat
-0.21
0.56
-1.67
...
-0.05
X^2
h^2
y^2
...
...
mat
0.445
0.987
-3.42
...
-0.06
X^T
h^T
y^T
y
$L(\hat{y}, y)$
W_{fc}, b_{fc}
$\frac{\partial L}{\partial W_{fc}}$
$\frac{\partial L}{\partial b_{fc}}$


a cat is sitting
on the mat
h^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h^1
y^1
$\frac{\partial L}{\partial y^1}$
cat
-0.21
0.56
-1.67
...
-0.05
X^2
h^2
y^2
$\frac{\partial L}{\partial y^2}$
...
...
mat
0.445
0.987
-3.42
...
-0.06
X^T
h^T
y^T
$\frac{\partial L}{\partial y^T}$
W_{fc}, b_{fc}
y
$L(\hat{y}, y)$

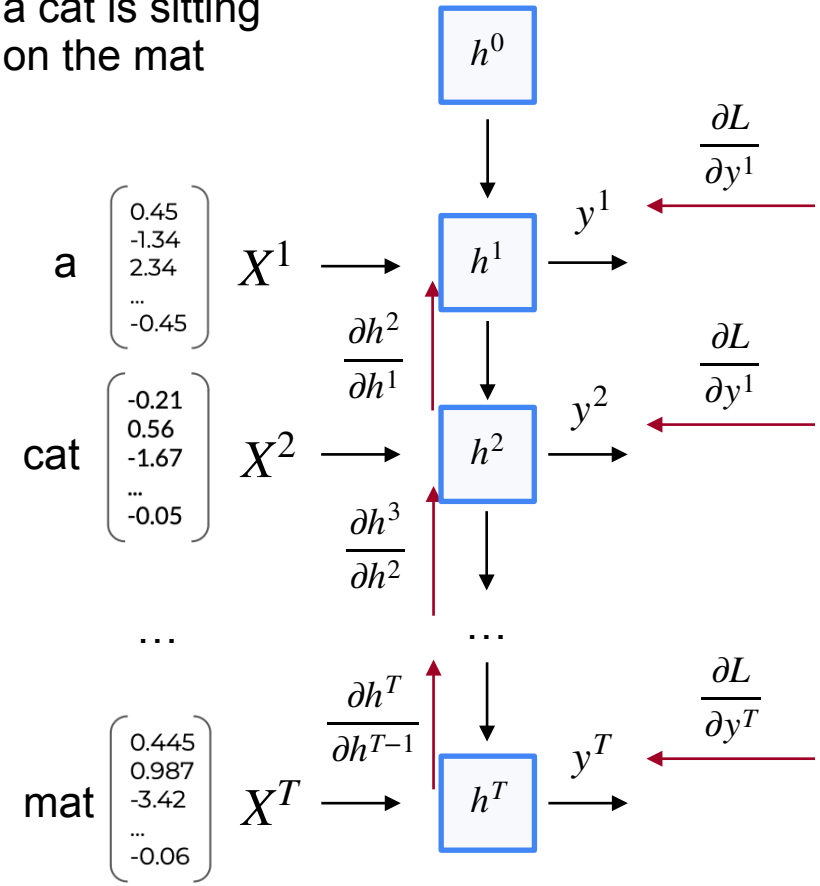


Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$

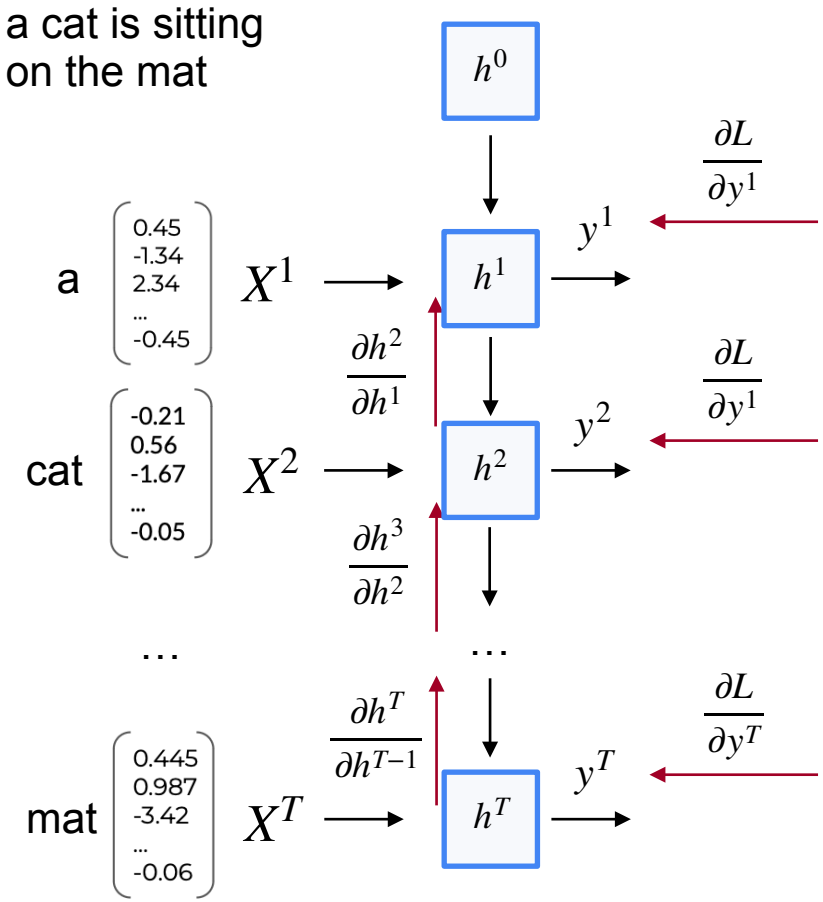
a cat is sitting
on the mat



Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$



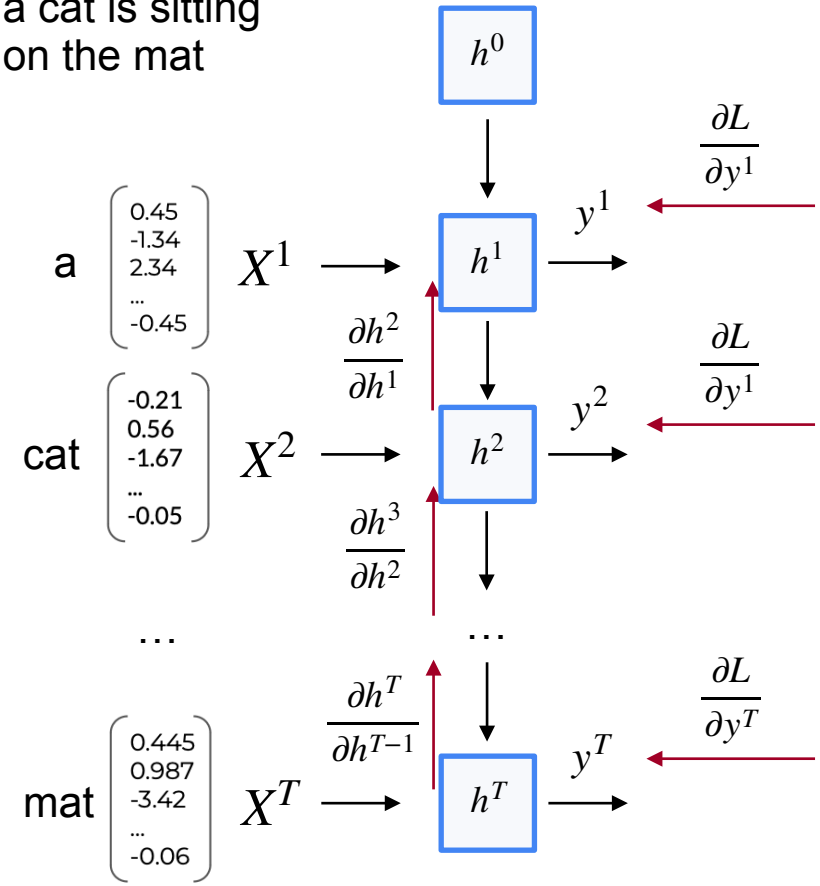
Hidden state on timestamp $t+1$ depends on hidden state on timestamp t

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

So $\frac{\partial L}{\partial h^t}$ is derived from all the $\frac{\partial L}{\partial h^{t+i}}$

U, W, b_h depend on all the h^t, so to get $\frac{\partial L}{\partial U}, \frac{\partial L}{\partial W}, \frac{\partial L}{\partial b_h}$, we need to propagate gradients back through time

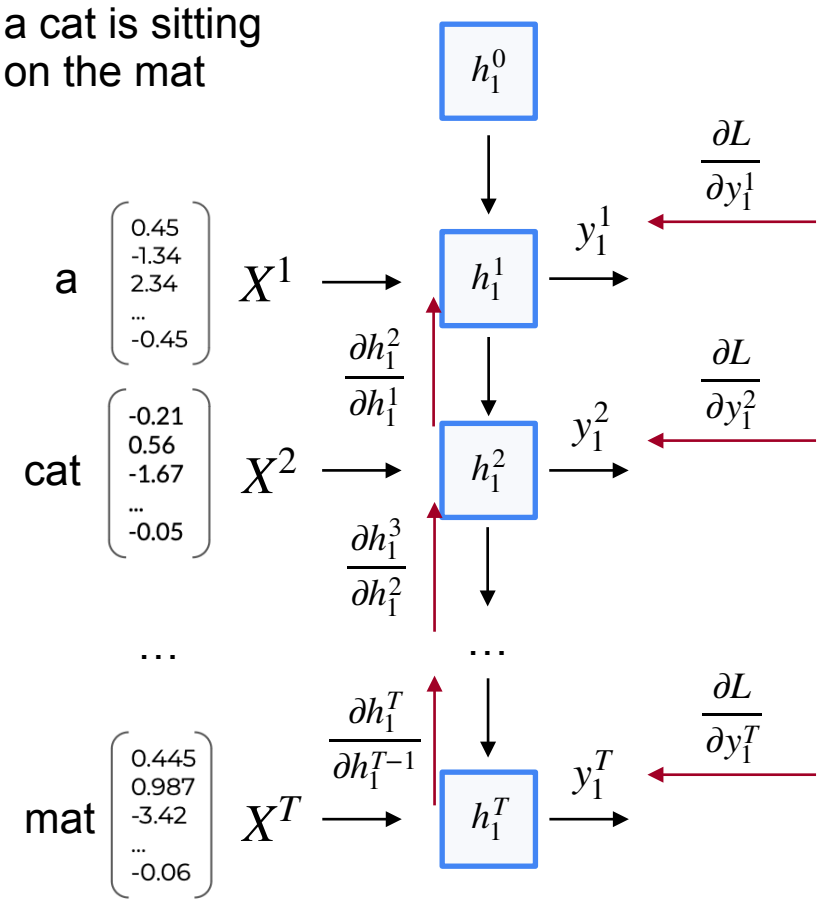
a cat is sitting on the mat



$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \dots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

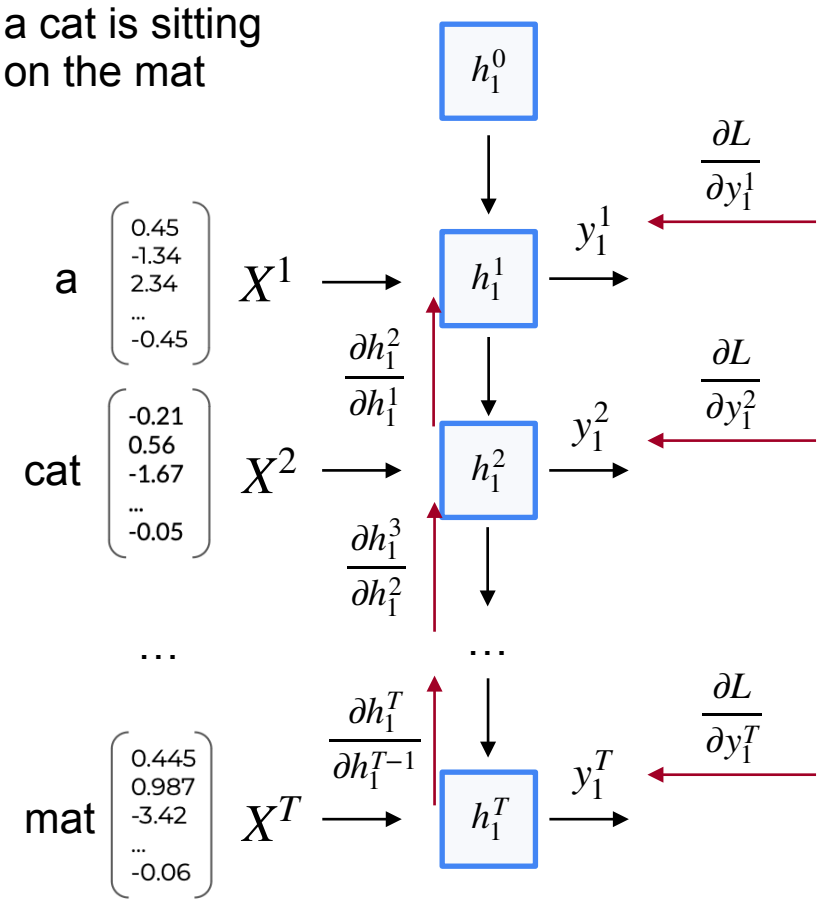
$$= \frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$



$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

$$= \frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$



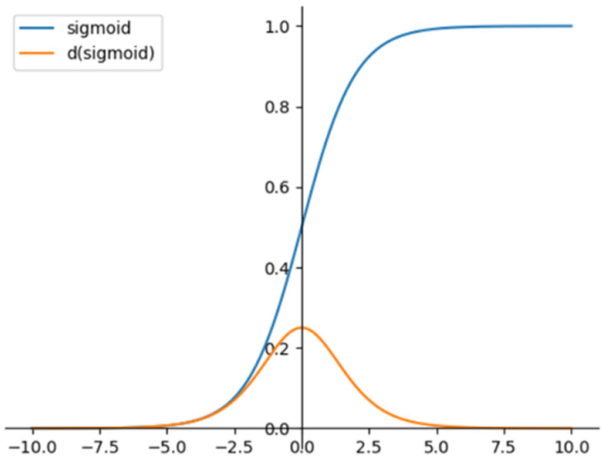
$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

$$=$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

There's derivative of σ in every multiplier



If we use *Sigmoid* as σ,
we risk to get a *vanishing gradient* problem

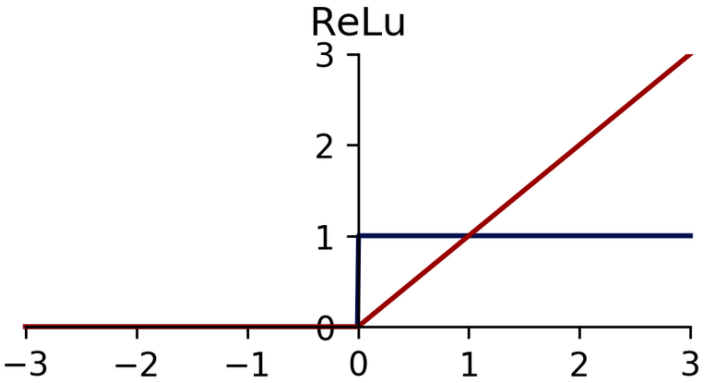
$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

$$=$$

$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

There's derivative of σ
in every multiplier



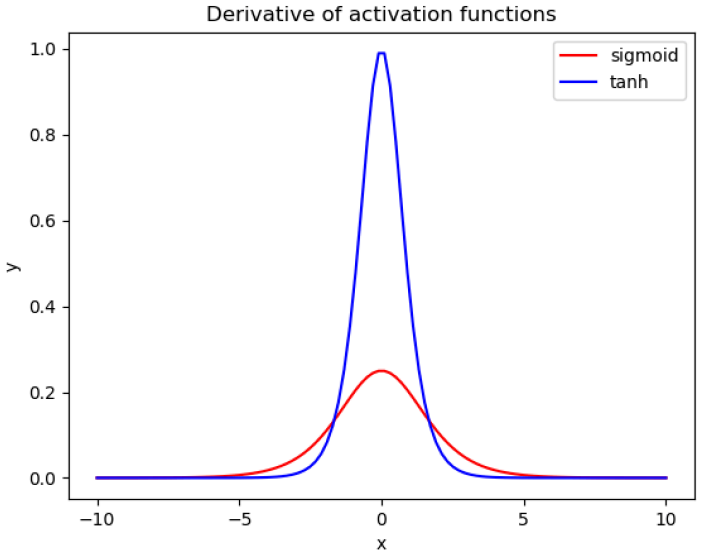
If we use *ReLU* as σ,
we risk to get an *exploding gradient* problem

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

$$= \frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

There's derivative of σ
in every multiplier



Compromise is using *Tanh* as σ

$$h^{t+1} = \sigma(WX^{t+1} + Uh^t + b_h)$$

$$\frac{\partial L}{\partial U} = \frac{\partial L}{\partial h^1}\frac{\partial h^1}{\partial U} + \frac{\partial L}{\partial h^2}\frac{\partial h^2}{\partial U} + \ldots + \frac{\partial L}{\partial h^T}\frac{\partial h^T}{\partial U}$$

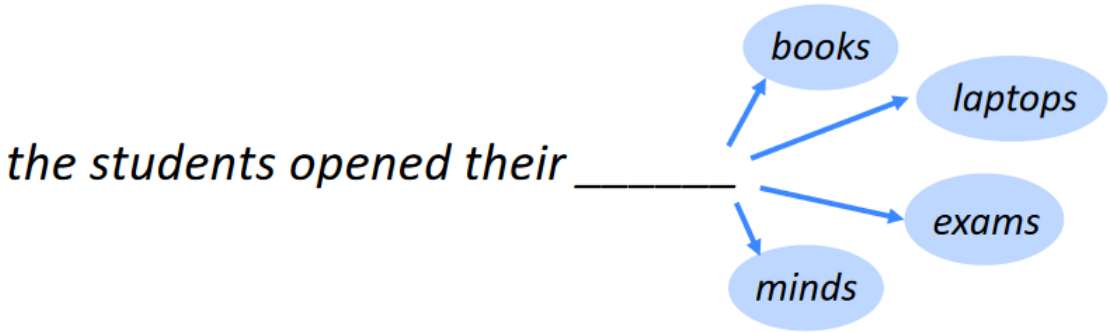
$$=$$

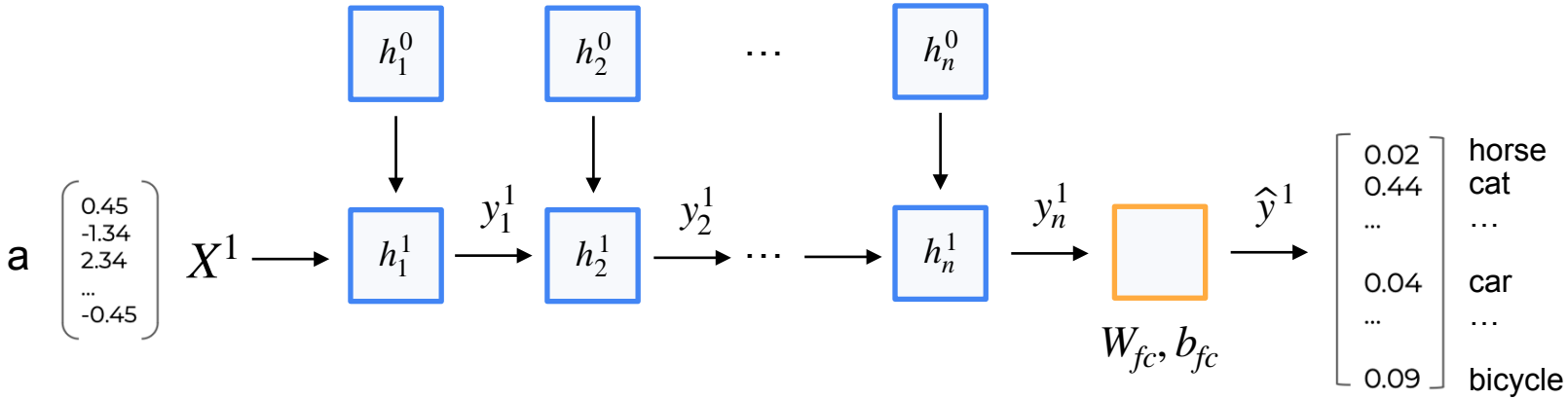
$$\frac{\partial h^T}{\partial h^{T-1}}\frac{\partial h^{T-1}}{\partial h^{T-2}}\cdots\frac{\partial h^2}{\partial h^1}\frac{\partial h^1}{\partial U}$$

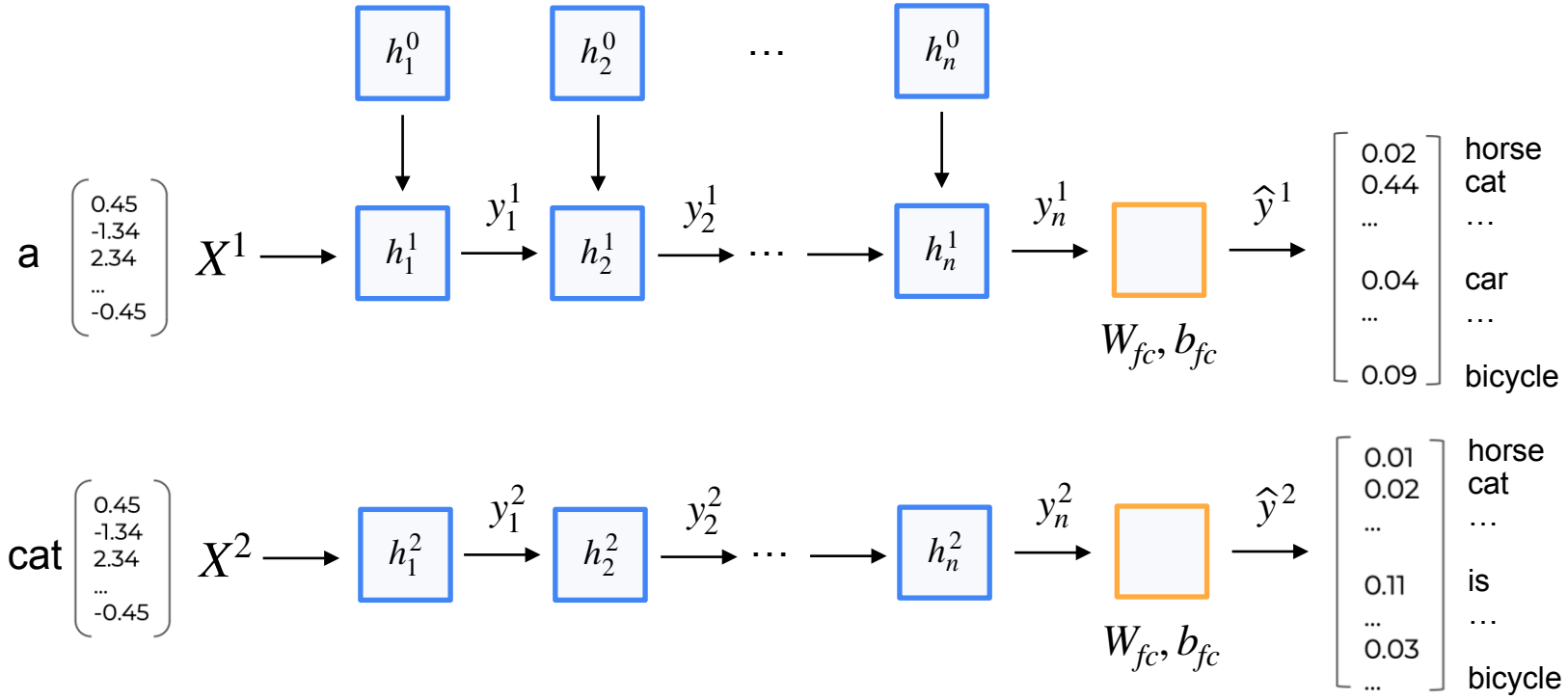
There's derivative of σ in every multiplier

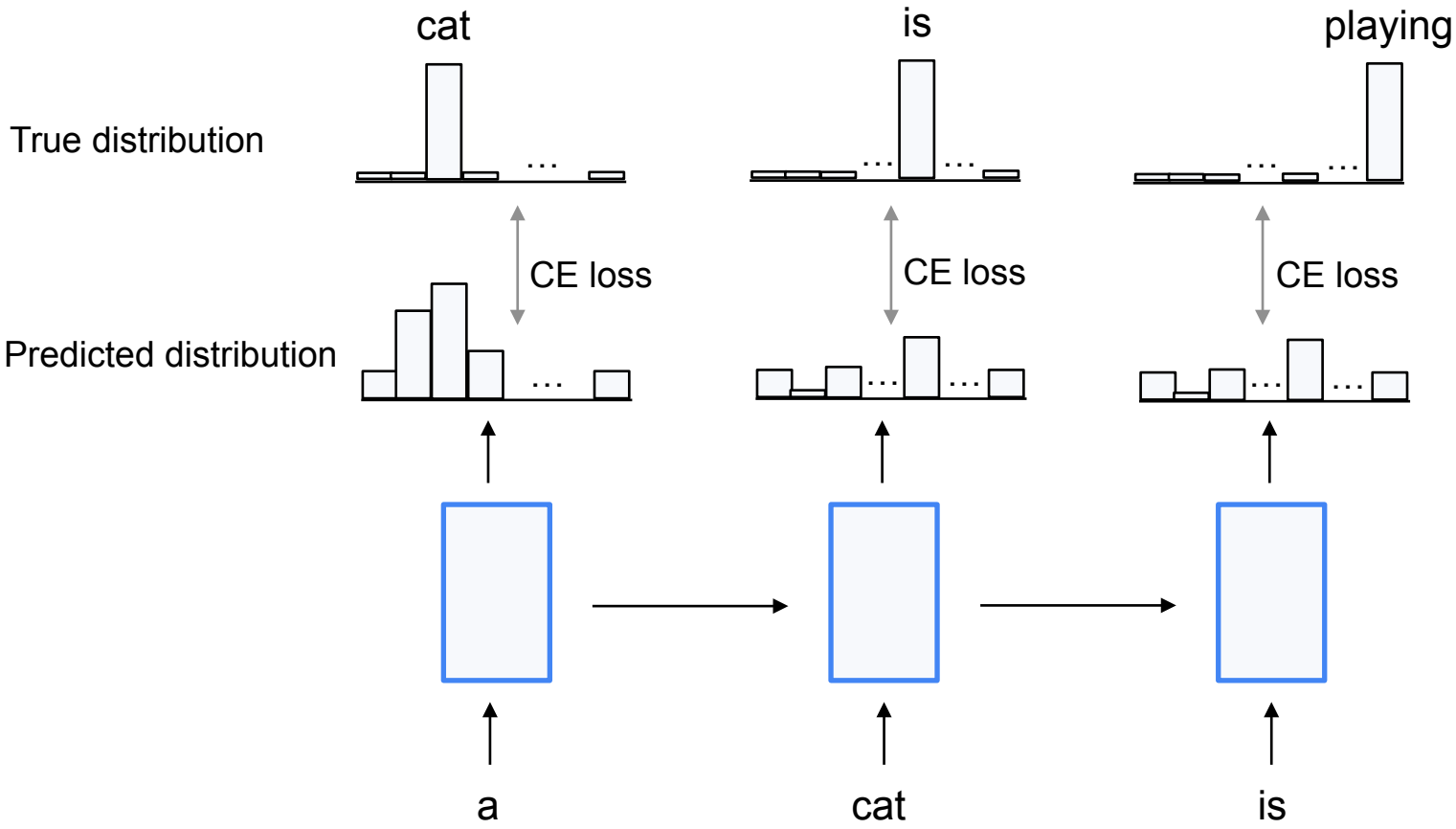
Language Modelling

Language Modelling is the task of modelling the probability of the next token




a
0.45
-1.34
2.34
...
-0.45
X^1
h_1^0
h_2^0
h_n^0
h_1^1
h_2^1
h_n^1
y_1^1
y_2^1
y_n^1
W_{fc}, b_{fc}
$\hat{y}^1$
0.02
0.44
...
0.04
...
0.09
horse
cat
car
bicycle


h_1^0
h_2^0
h_n^0
a
0.45
-1.34
2.34
...
-0.45
X^1
h_1^1
y_1^1
h_2^1
y_2^1
h_n^1
y_n^1
W_{fc}, b_{fc}
$\hat{y}^1$
0.02
0.44
...
0.04
...
0.09
horse
cat
...
car
...
bicycle
cat
0.45
-1.34
2.34
...
-0.45
X^2
h_1^2
y_1^2
h_2^2
y_2^2
h_n^2
y_n^2
W_{fc}, b_{fc}
$\hat{y}^2$
0.01
0.02
...
0.11
...
0.03
...
horse
cat
...
is
...
bicycle

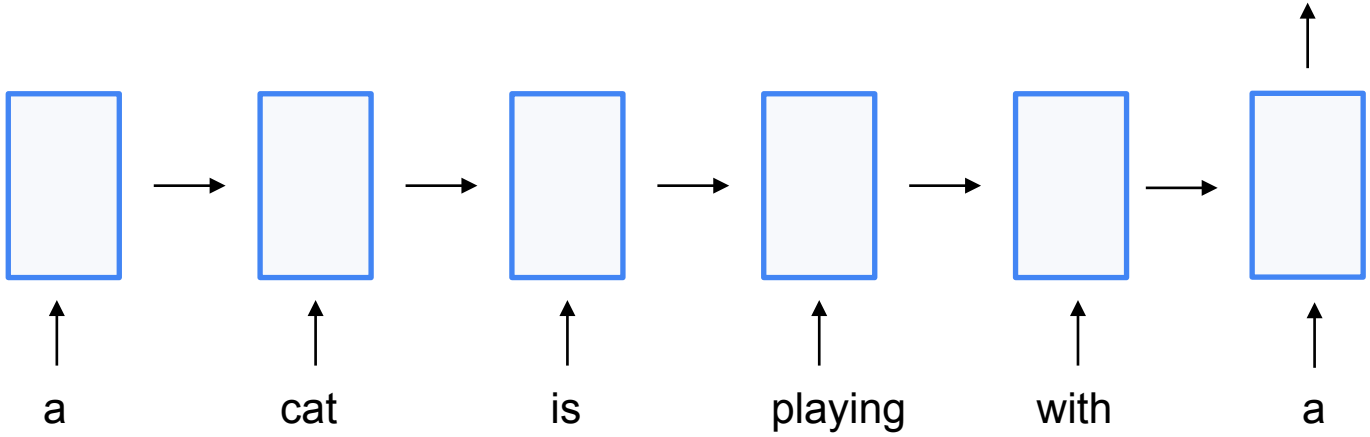

cat
is
playing
True distribution
CE loss
CE loss
CE loss
Predicted distribution
a
cat
is

RNN hidden state update:

$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

This hidden state of RNN should contain all the information about sentence

RNNs are prone to forgetting

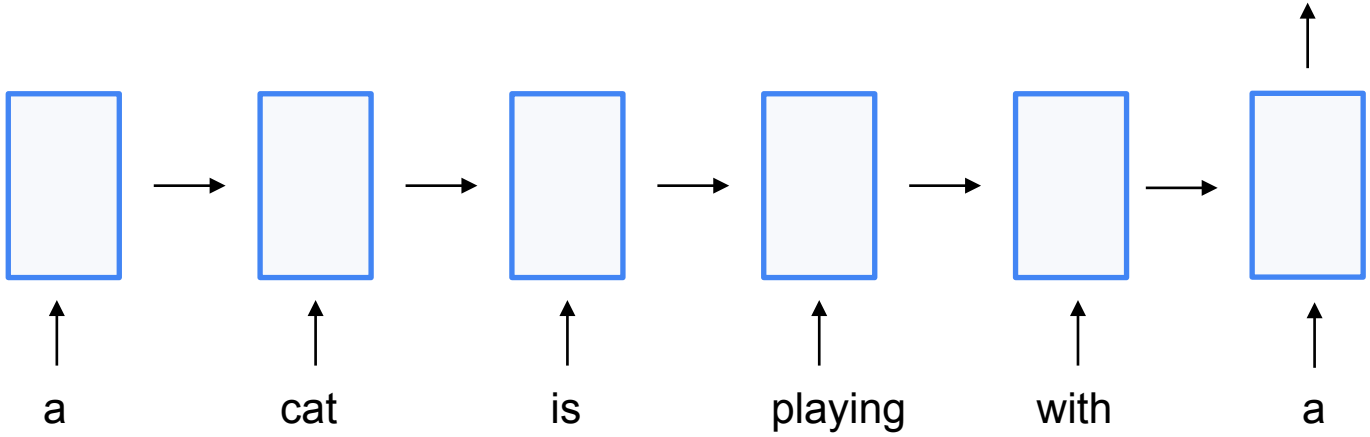


RNN hidden state update:

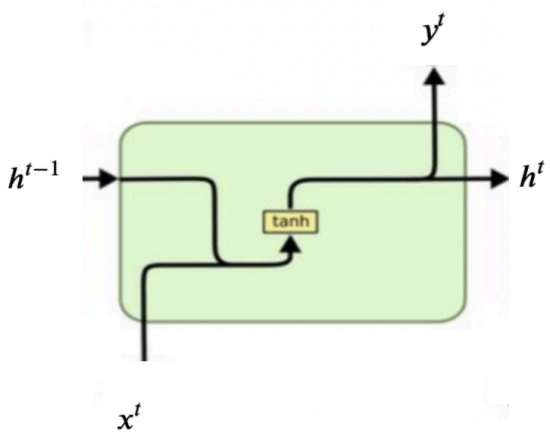
$$h^t = \sigma(WX^t + Uh^{t-1} + b_h)$$

This hidden state of RNN should contain all the information about sentence

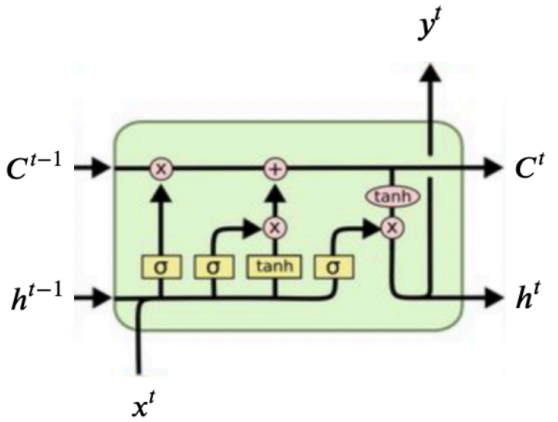
RNNs are prone to forgetting



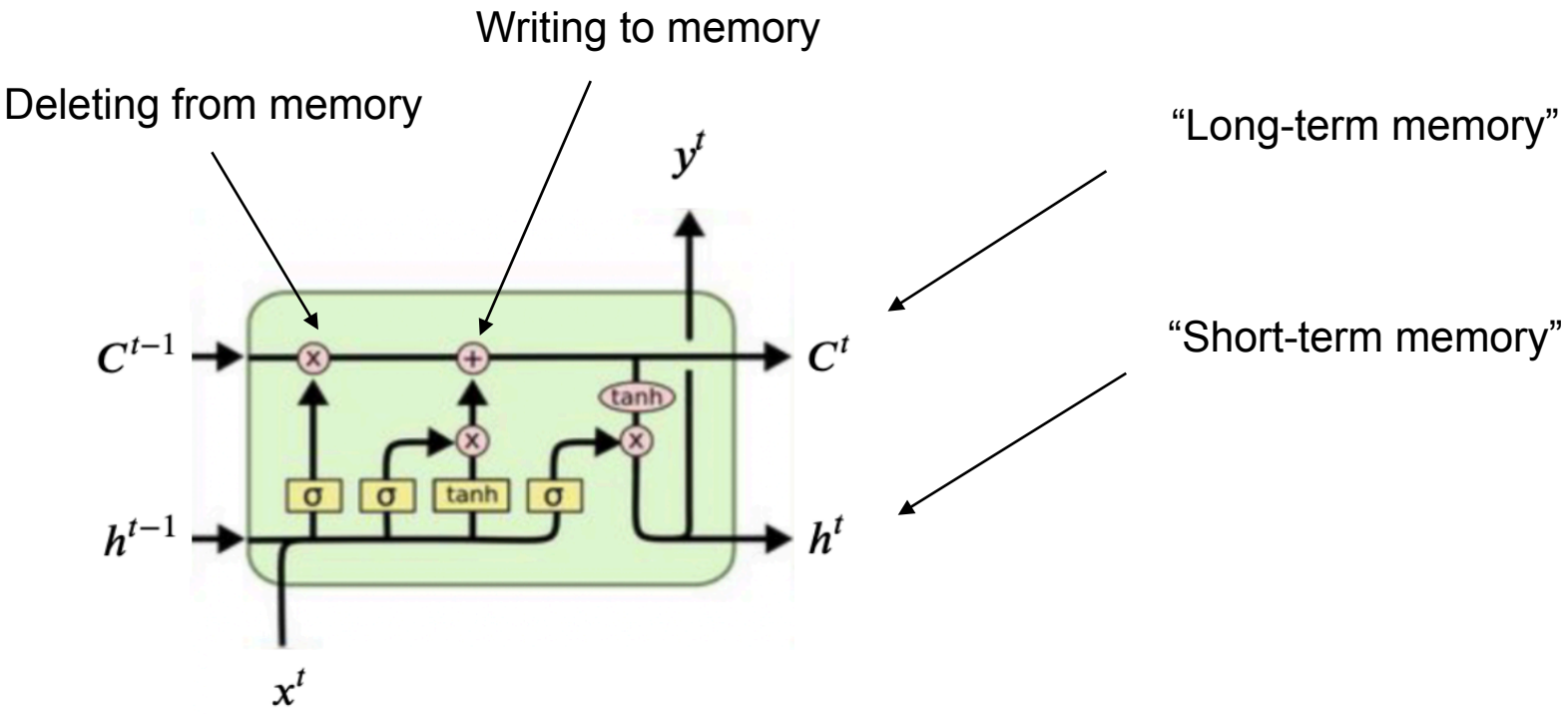
Vanilla RNN



LSTM (Long Short-Term Memory)



LSTM


Writing to memory
Deleting from memory
y^t
C^{t-1}
C^t
h^{t-1}
h^t
x^t
σ
σ
tanh
σ
tanh
“Long-term memory”
“Short-term memory”

LSTM

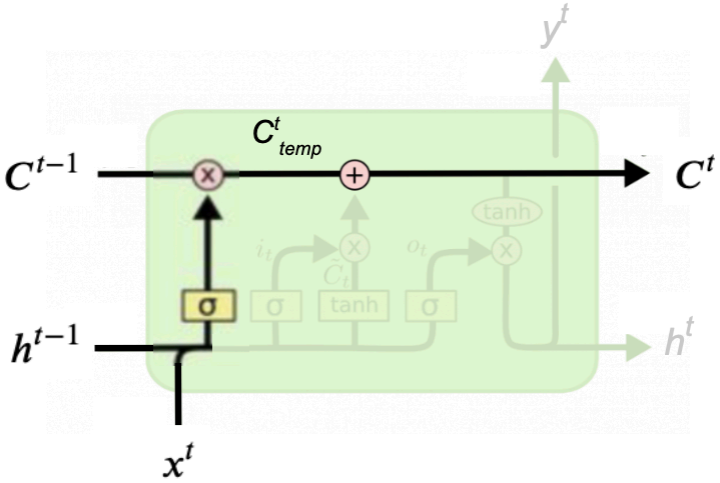
LSTM uses **gating mechanism**

Calculate, how much we want to forget from each of positions of C
It is a vector of elements in range [0, 1]

Sigmoid function

Forget gate:

$$f^t = \sigma(W_f \cdot [h^{t-1}, x^t] + b_f)$$

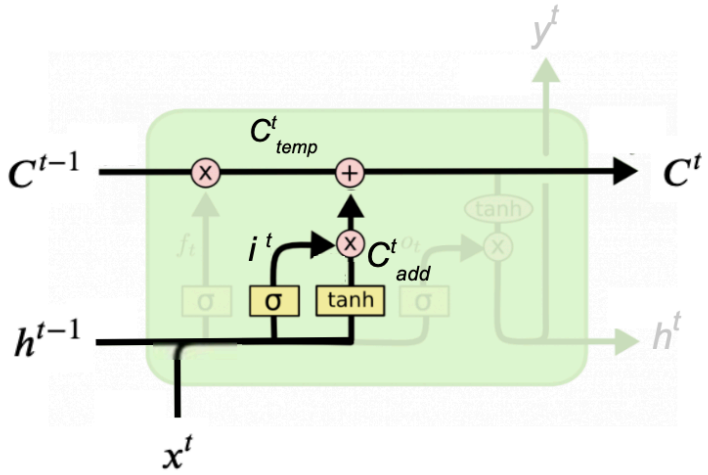


$$C^t_{temp} = f^t * C^{t-1}$$

Actually “forgetting”

LSTM

LSTM uses **gating mechanism**



Calculate, *how much* we want to add to C
It is a vector of elements in range [0, 1]

Sigmoid function

Input gate:

$$i^t = \sigma(W_i \cdot [h^{t-1}, x^t] + b_i)$$

Calculate, *what* we want to add to C

$$C^t_{add} = tanh(W_C \cdot [h^{t-1}, x^t] + b_C)$$

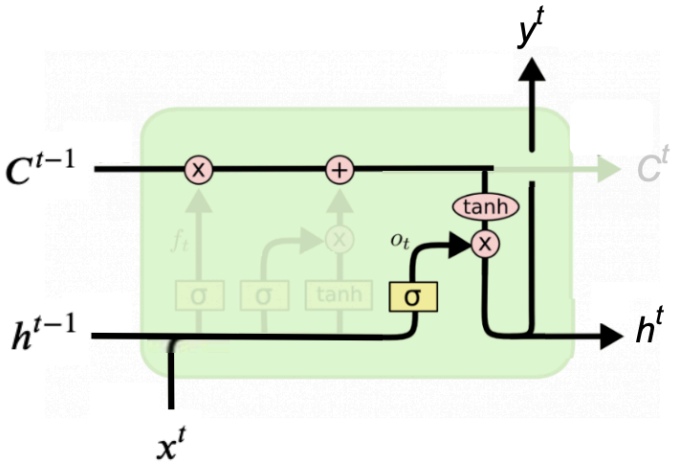
$$C^t = C^t_{temp} + i^t * C^t_{add}$$

Adding new information

LSTM

Calculate, how much we want to copy from C to h
It is a vector of elements in range [0, 1]

Updating h^t and calculating output y^t



$$o^t = \sigma(W_o \cdot [h^{t-1}, x^t] + b_o)$$

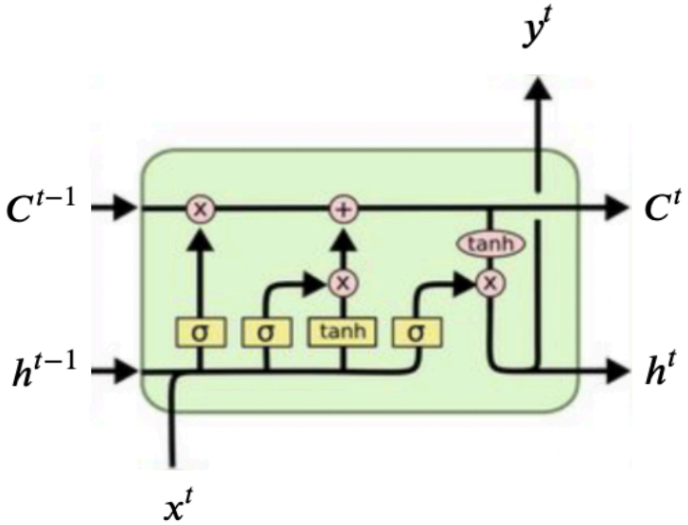
Copying from C to h

$$h^t = o^t * tanh(C^t)$$

Calculating output

$$y^t = \sigma(W_y h^t + b_y)$$

LSTM



$$f^t = \sigma(W_f \cdot [h^{t-1}, x^t] + b_f)$$

$$i^t = \sigma(W_i \cdot [h^{t-1}, x^t] + b_i)$$

$$C^t_{add} = tanh(W_C \cdot [h^{t-1}, x^t] + b_C)$$

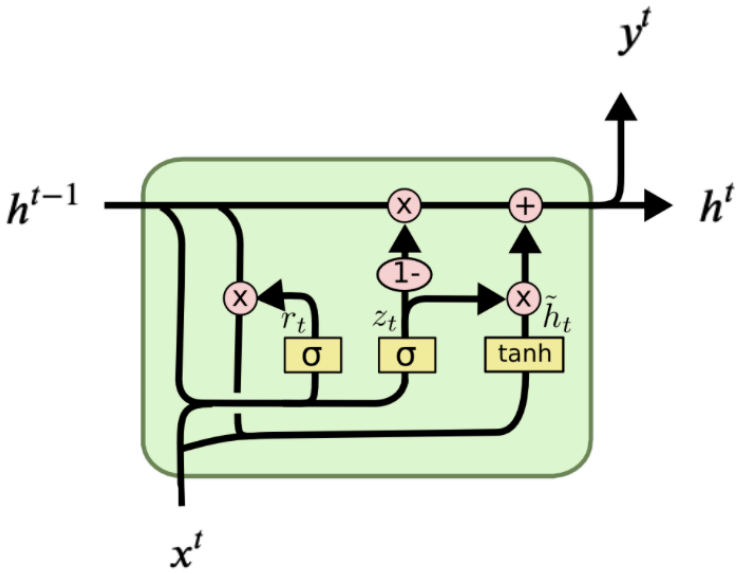
$$C^t = f^t * C^{t-1} + i^t * C^t_{add}$$

$$o^t = \sigma(W_o \cdot [h^{t-1}, x^t] + b_o)$$

$$h^t = o^t * tanh(C^t)$$

$$y^t = \sigma(W_y h^t + b_y)$$

GRU



GRU is a “light version” of LSTM